The Full Trading System

Build, Deploy, and Operate an Autonomous Weather Trading Operation on Polymarket

By Obsidian — CEO, JSJ Consulting

Version 1.0 — March 2026

> "We lost $300 in one trade because we didn't account for rounding. We made it back because we built a system that accounts for everything. This is that system — the complete blueprint, the code, the failures, and the fixes."

© 2026 JSJ Consulting. All rights reserved.

This guide is licensed for personal use only. Do not redistribute, resell, or share without written permission from JSJ Consulting.

Table of Contents

- Part 1: Introduction — This Is the Complete System
- Part 2: Full System Architecture
- Part 3: The Meteorological Edge — Deep Dive
- Part 4: The Gate Framework — Complete Rule Set
- Part 5: The Cusp Model — Full Mathematical Treatment
- Part 6: OpenClaw Setup — Advanced Configuration
- Part 7: The Python Trading Bot — Installation & Setup
- Part 8: Wethr.net Pro API Integration
- Part 9: METAR Data Streaming & Monitoring
- Part 10: Daily Automation Workflows
- Part 11: The Decision Database — Schema & Pattern Analysis
- Part 12: The Complete Post-Mortem — Real Trades, Real Losses, Real Fixes
- Part 13: Advanced Monitoring & Alerts
- Part 14: Scaling & Optimization
- Part 15: Reference Appendices

Part 1: Introduction — This Is the Complete System

You read the Starter Guide. You set up OpenClaw. You have an AI analyst on your phone delivering morning briefings and responding to research questions in Telegram. You've been watching weather markets on Polymarket, maybe placed a few small trades, maybe not.

Now you want the engine.

This guide is the engine. Not prompts — code. Not a framework you interpret — a system that runs. Not general concepts about weather markets — the specific mathematical models, gate rules, daemon processes, and database schemas that turned a 26.7% win rate into a profitable trading operation.

Here's what changed between February 15 (the day we lost $300 on a single rounding error) and the system you're about to build:

Before	After
Manual temperature checking	Sub-15-second METAR detection daemon
Eyeballing safety margins	WU rounding calculator with 0.1°F precision
Gut-feel position sizing	Kelly Criterion with margin-based tiering
No trade-blocking logic	Four weather gates with hard kill switches
No decision logging	SQLite database with pattern extraction queries
Hope-based monitoring	Real-time SSE push stream with Telegram alerts

The Starter Guide gave you the analyst. This guide gives you the trading operation.

What You're Building

By the end of this guide, you will have:

1. **A Python trading system** that scans Polymarket weather markets, detects mispriced brackets, and calculates exact edges using multiple data sources
2. **Real-time METAR monitoring** with sub-10-second detection latency — 30–60 minutes faster than most market participants
3. **A cusp detection model** that identifies when Weather Underground rounding creates exploitable edges invisible to other traders
4. **Weather pattern gates** that automatically block trades in known failure scenarios (sea breeze caps, marine layers, forecast divergence, polar vortex locks)
5. **A position sizer** implementing modified half-Kelly with safety margin tiering that prevents the exact mistake that cost us $300
6. **A decision database** that logs every trade decision and enables pattern extraction across hundreds of trades
7. **Automated daily workflows** via cron, tmux, and OpenClaw heartbeats that run the entire operation with minimal manual intervention
8. **Wethr.net Pro API integration** — the resolution-matching data source that tells you exactly what Weather Underground will display

Who This Is For

You've completed the Starter Guide. You're comfortable running terminal commands. You can follow Python code even if you don't write it daily. You have `pip install` muscle memory.

You want to go from “AI analyst giving me briefings” to “automated trading operation with code I understand and control.”

You don’t need a CS degree. You do need patience — this is a real system with real components that need to be configured correctly. The setup takes 2–3 hours. The payoff is a trading operation that runs on autopilot.

Infrastructure Costs

Item	Monthly Cost	Required?
Wethr.net Pro API	~$25	Yes — resolution-matching data
Anthropic API (Claude)	~$21 at typical usage	Yes — fair value estimation
Polymarket USDC	Your trading capital	Yes — start with $100–500
OpenClaw	Free (self-hosted)	Yes — from your Starter Guide setup
VPS (optional)	$4–5	No — runs fine on your laptop

Total overhead: ~$50/month. The math works at even small position sizes — 2–4 trades per day with 8–25% average edge — but only if the system is working correctly. That’s why we document the failures alongside the wins.

The Honest Numbers

After the February audit and subsequent fixes:

- **15 initial trades (Feb 15–18):** 26.7% win rate. Four wins, eleven losses. Terrible.
- **Root causes identified:** WU rounding blindness (3 trades), panic interpretation (3), pyramiding (2), wrong station (1), sea breeze (1), forecast divergence (1)
- **Components built from each failure:** WU Rounding Calculator, Position Sizer, Weather Gates, Signal Validator, METAR Daemon, Station Map
- **Post-fix qualifying trade characteristics:** Early entry, held to resolution, METAR confirmation, 2+ data sources
- **Chicago:** 100% win rate (2/2), the only reliable city in early data
- **Confidence paradox discovered:** “High confidence” trades lost more than “medium confidence” trades — because high confidence led to larger positions, amplifying losses

This isn’t a get-rich-quick system. It’s an edge-exploitation framework with honest accounting. Let’s build it.

Part 2: Full System Architecture

The Complete Stack

YOU (Trader)

Review opportunities • Approve/reject trades • Set risk parameters
Monitor P&L • Adjust gates and thresholds • Override when warranted

Interface: Telegram (phone) + Terminal (laptop) + Dashboard (optional)

Telegram messages + CLI commands

AI AGENT (OpenClaw)

Heartbeat (30 min)	Morning Brief (8:00 AM EST)	Real-time Alert Engine (Push → Telegram)
• Fetch METAR	• NWS forecasts	• New high/low detection
• Run gates	• WU overnight	• Safety margin breach
• Cusp model	• Top 3 opps	• Gate trigger after entry
• Update P&L	• Open position status	• Suspect temp rejection
• Log to DB		• Forecast revision

TRADING FRAMEWORK (Sequential Pipeline)

GATE CHECK	CUSP MODEL	POSITION SIZER
Sea breeze Marine layer Forecast div Polar vortex	T-group parse Oscillation detection WU rounding margin calc	Kelly criterion (half-Kelly) Margin tiering Time-of-day penalties Portfolio limits
BLOCKED	INSUFFICIENT	$0
TRADE KILLED	TRADE KILLED	TRADE KILLED

Three independent kill switches. Any one fires → no trade.

```
DECISION DATABASE (SQLite)
Every decision logged • Pattern extraction • Performance tracking
```

```
Wethr.net Pro          METAR / ASOS           Polymarket
API                    (aviationweather       Gamma API
                        .gov)
REST Endpoints:                               REST Endpoints:
• /wethr-high          Raw METAR obs:         • /markets
  (WU-logic)           • Every ~5 min         • /prices
• /nws-forecast        • T-group: 0.1°C       • /book
• /all-highs           • Wind, clouds         • /trades
                       • Pressure
SSE Push Stream:                              Resolution:
• new_high             SPECI (special):       • WU displayed
• new_low              • On significant         high temp
• observation            weather change       • Specific ICAO
• suspect_temp                                  station
• nws_update           Aggressive poll:
                       • 2s in :52-:58
                         UTC window
```

Data Flow: From METAR Observation to Trade Decision

When a new METAR observation posts (typically at :53 past the hour UTC), here's what happens in the system — all automated:

```
:53:00 UTC    METAR posts to aviationweather.gov

:53:02        metar_aggressive_daemon.py detects new T-group
              (polling every 2 seconds during :52-:58 window)

:53:03        Parse T-group → precise temperature
              Example: T01440044 → 14.4°C → 57.92°F

:53:04        Run WU rounding calculator:
                57.92°F rounds to 58°F on Weather Underground
                If holding 56-57°F bracket → EXIT ALERT
                If holding 58-59°F bracket → HOLD (margin = 1.08°F)

:53:05        Run all weather gates for affected cities:
```

```
                  Sea breeze: N/A (not Miami, or wind not matching)
                  Marine layer: N/A (not Seattle/NYC coastal)
                  Forecast divergence: Scanner and actual aligned
                  If any gate fires → KILL ALERT

:53:08        Cusp model recalculates safety margins:
                - Current margin to bracket boundaries
                - Trend analysis (warming/cooling/plateau)
                - Oscillation detection
                - Risk assessment

:53:10        If margin dropped below threshold → EXIT ALERT (Telegram)
              If new opportunity detected → OPPORTUNITY ALERT with sizing

:53:12        Decision logged to SQLite (decisions.db)

:53:15        Wethr.net Push API confirms WU-logic high update
              (provides resolution-matching confirmation)
```

Total latency from METAR post to trading decision: under 15 seconds.

Most Polymarket weather traders check the Weather Channel app or weather.com. Those sources update every 30–60 minutes. You're seeing the actual observation 30–60 minutes before they do. That's the edge.

Component Dependency Map

Understanding which components depend on which helps when debugging:

```
wethr_client.py     best_trades_query.py     best_trades.py


                      trade_executor.py     live_trader.py


metar_fetcher.py      signal_validator.py


metar_cusp_model.py     metar_aggressive_daemon.py


wu_rounding.py     position_sizer.py


weather_gates.py

polymarket_fetcher.py     best_trades_query.py
```

```
trend_analysis.py
```

Key insight: `wu_rounding.py` and `weather_gates.py` are the two safety layers. Everything flows through them. If either is broken, the system is unsafe. That's why both include self-validation tests that run on import.

Part 3: The Meteorological Edge — Deep Dive

How Weather Markets Resolve: The Precise Mechanics

Every Polymarket weather market specifies a resolution source. For the six cities we trade, resolution uses **Weather Underground's displayed high temperature** for a specific airport METAR station.

This is critical to understand precisely. It's not: - The raw METAR temperature (that's in Celsius, with different precision) - The NWS official high (that uses different logic) - The Weather Channel high (different data processing) - AccuWeather (different source entirely)

It is specifically **what the Weather Underground website displays** after applying its own rounding and processing to the METAR data stream.

The Station Map — Get This Wrong and Nothing Else Matters:

City	ICAO Station	Airport	WU Page
Atlanta	KATL	Hartsfield-Jackson Intl	wunderground.com/...KATL
Dallas	KDAL	Dallas Love Field	wunderground.com/...KDAL
Miami	KMIA	Miami Intl	wunderground.com/...KMIA
Seattle	KSEA	Seattle-Tacoma Intl	wunderground.com/...KSEA
New York City	KLGA	LaGuardia	wunderground.com/...KLGA
Chicago	KORD	O'Hare Intl	wunderground.com/...KORD

Dallas is KDAL (Love Field), not KDFW (DFW Airport). These stations can differ by 3–5°F on the same day due to urban heat island effects, elevation differences, and microclimates. We learned this the expensive way — a Dallas trade failed in February because we initially pulled data from the wrong station. One letter in a four-letter code. $50 gone.

METAR: The Primary Signal

METAR (Meteorological Aerodrome Report) is the lingua franca of aviation weather. Every staffed airport weather station produces METAR observations, typically every hour at :53 past the hour UTC, with special reports (SPECI) issued when conditions change rapidly.

A complete METAR observation decoded:

```
KATL 151655Z 10005KT 10SM FEW250 14/04 A3012 RMK AO2 SLP203 T01440044 10183 20122 53007
```

Field	Raw	Decoded	Trading Relevance
Station ID	`KATL`	Atlanta Hartsfield-Jackson	Verify this matches your target
Timestamp	`151655Z`	15th day, 16:55 UTC	Convert to local: 11:55 AM EST
Wind	`10005KT`	From 100° at 5 knots	Gate triggers: sea breeze, marine layer
Visibility	`10SM`	10 statute miles	Clear — not directly relevant
Sky condition	`FEW250`	Few clouds at 25,000 ft	Cloud breaks = warming risk
Temp/Dewpoint	`14/04`	14°C / 4°C (whole degrees)	**Low precision — do not use for trading**
Altimeter	`A3012`	30.12 inHg	Pressure trends = weather pattern shifts
T-group	`T01440044`	**14.4°C / 0.44°C**	**THE MONEY FIELD — 0.1°C precision**
6-hr Max	`10183`	Max temp: 18.3°C	Historical peak reference
6-hr Min	`20122`	Min temp: 12.2°C	Overnight low reference
Pressure trend	`53007`	Rising 0.7 mb in 3 hours	Clearing = warming potential

The T-Group: Why 0.1°C Precision Changes Everything

The main temperature field in METAR (`14/04`) rounds to whole Celsius degrees. That's a 1.8°F uncertainty range. Useless for trading brackets that resolve on single Fahrenheit degrees.

The T-group in the remarks section gives you **ten times the precision**:

```
T-group format:  T [sign][temp×10] [sign][dewpoint×10]

T 0 144 0 044

            Dewpoint: 4.4°C
           Dewpoint sign: 0 = positive
          Temperature: 14.4°C
         Temperature sign: 0 = positive, 1 = negative
         T-group identifier
```

Parsing in Python:

```python
def parse_tgroup(tgroup_str: str) -> dict:
    """
    Parse METAR T-group for precise temperature.
```

```
    Example: "T01440044" → {'temp_c': 14.4, 'temp_f': 57.92, 'dewpoint_c': 4.4}

    The T-group is the ONLY field with enough precision for weather trading.
    The main temp field (14/04) loses 0.1°C resolution - a 0.18°F error
    that can flip a WU rounding result.
    """
    if not tgroup_str or len(tgroup_str) != 9 or tgroup_str[0] != 'T':
        return None

    # Temperature
    temp_sign = 1 if tgroup_str[1] == '0' else -1
    temp_c = temp_sign * int(tgroup_str[2:5]) / 10.0

    # Dewpoint
    dew_sign = 1 if tgroup_str[5] == '0' else -1
    dewpoint_c = dew_sign * int(tgroup_str[6:9]) / 10.0

    # Precise Fahrenheit conversion
    temp_f = temp_c * 9.0 / 5.0 + 32.0

    return {
        'temp_c': temp_c,
        'temp_f': round(temp_f, 2),
        'dewpoint_c': dewpoint_c,
        'wu_display': round(temp_f),  # What WU will show
        'raw': tgroup_str
    }


# Examples that show why 0.1°C matters:
examples = [
    "T01440044",  # 14.4°C = 57.92°F → WU: 58°F
    "T01390044",  # 13.9°C = 57.02°F → WU: 57°F
    "T01420044",  # 14.2°C = 57.56°F → WU: 58°F
    "T01410044",  # 14.1°C = 57.38°F → WU: 57°F
]

for tg in examples:
    result = parse_tgroup(tg)
    print(f"  {tg}: {result['temp_c']}°C = {result['temp_f']}°F → WU displays {result['wu_displ
```

Output:

```
  T01440044: 14.4°C = 57.92°F → WU displays 58°F
  T01390044: 13.9°C = 57.02°F → WU displays 57°F
  T01420044: 14.2°C = 57.56°F → WU displays 58°F
  T01410044: 14.1°C = 57.38°F → WU displays 57°F
```

The difference between `T01410044` (57.38°F → WU shows 57) and `T01420044` (57.56°F

→ WU shows 58) is **0.1°C** — about 0.18°F. That's the difference between winning and losing a bracket bet. The whole-degree METAR field would show "14°C" for both. Only the T-group tells them apart.

NWS Forecasts: The Prediction Layer

The National Weather Service issues forecast updates every 1–3 hours using the National Blend of Models (NBM), which incorporates data from GFS, HRRR, NAM, ECMWF, and other numerical weather prediction models.

For trading, NWS forecasts serve three purposes:

1. **Pre-market thesis generation:** Before METAR data reaches trading-relevant brackets, the forecast tells you where temperature is likely going
2. **Edge window identification:** When NWS revises a forecast (e.g., updates Atlanta's projected high from 62°F to 65°F at 11 AM), the market hasn't repriced yet — most traders check forecasts once in the morning
3. **Trend validation:** Comparing NWS hourly projections against actual METAR readings reveals whether the forecast is tracking reality

Reading the NWS hourly profile:

```python
from src.wethr_client import WethrClient

client = WethrClient()
forecast = client.get_nws_forecast('KATL')

print(f"Forecast High: {forecast['high']}°F")
print(f"Forecast Low:  {forecast['low']}°F")
print(f"Version:       v{forecast['version']}")

# Hourly temps (index 0 = midnight local, 12 = noon, etc.)
hourly = forecast['hourly_temps']
print(f"\nHourly profile:")
for hour in range(6, 22):  # 6 AM to 10 PM
    print(f"  {hour:02d}:00 → {hourly[hour]}°F")
```

```
Forecast High: 65°F
Forecast Low:  45°F
Version:       v12

Hourly profile:
  06:00 → 46°F
  07:00 → 47°F
  08:00 → 49°F
  09:00 → 53°F
  10:00 → 57°F
  11:00 → 60°F
  12:00 → 63°F
  13:00 → 65°F    ← Projected peak
```

```
14:00 → 65°F    ← Sustained peak
15:00 → 64°F    ← Beginning decline
16:00 → 62°F
17:00 → 60°F
18:00 → 57°F
19:00 → 55°F
20:00 → 53°F
21:00 → 52°F
```

The forecast version matters. Each NWS update increments the version. If the 8 AM forecast (v10) said 65°F and the noon update (v12) says 67°F, that's a 2°F upward revision. The market is still pricing the old forecast. You have a 30–60 minute window before other traders notice the revision.

Weather Underground: The Resolution Engine

WU is the resolution oracle. What WU displays as the daily high is what determines whether your bracket bet wins or loses. Understanding WU's processing is non-negotiable.

WU's temperature processing chain:

```
Raw METAR T-group (°C, 0.1° precision)


Convert to Fahrenheit: temp_f = temp_c × 9/5 + 32


Standard rounding: round(temp_f) to nearest integer


High tracking: if rounded > current daily high → update display


THIS is the resolution value
```

Critical WU behavior — the high only goes up:

Once WU displays a daily high, it doesn't go back down. If the 1 PM METAR shows 65°F and the 3 PM METAR shows 62°F, the WU daily high stays at 65°F. This means:

- If the current WU high is already in your bracket → you're winning (as long as it doesn't climb out the top)
- If the current WU high is below your bracket → you need future warming
- If the current WU high is above your bracket → you've already lost (the high can't decrease)

The Wethr.net API gives you the WU-logic high in real time. This is the closest you can get to reading the resolution value before resolution happens.

The Three-Source Confirmation Framework

Our February audit revealed that **every winning trade had confirmation from 2+ data sources. Every avoidable loss had only one.**

Trade Signal	Source 1	Source 2	Source 3 (Bonus)
"High is locked at 57°F"	METAR T-group confirms	Wethr.net WU high stable	NWS hourly shows decline
"Temperature will exceed 70°F"	NWS forecast updated upward	METAR trend: rising 2°C/hr	Wethr.net push: approaching
"Peak is behind us"	METAR plateau 30+ min	Trend analyzer shows cooling	NWS hourly shows decline
"Forecast is busted"	NWS says 78°F	METAR stuck at 74°F for 45 min	Divergence > 1.5°F

The rule is simple: no single-source trades. If you can't confirm from two independent sources, the signal isn't strong enough. This alone would have prevented 4 of our 11 February losses.

Station-Specific Meteorological Patterns

Each station has quirks that forecast models don't fully capture. These come from geography, urban heat effects, and microclimate patterns. Understanding them gives you an edge over traders who treat "temperature" as a uniform phenomenon.

Atlanta (KATL): - Late afternoon warming events possible even after rain stops — cloud breaks allow solar heating - Urban heat island at the airport creates a 1–2°F warm bias vs surrounding areas - Winter frontal passages can drop temperature 5–8°F in 30 minutes - The Feb 15 disaster was a late-afternoon warming event we didn't anticipate

Dallas (KDAL — Love Field): - Afternoon peaks typically 2–4 PM in winter, 3–5 PM in summer - Dry airmass amplifies diurnal range: 30°F+ swings common in spring - **KDAL ≠ KDFW** — Love Field is in the urban core, DFW Airport is 20 miles north. Temperatures can differ 3–5°F - Wind shifts from south to north indicate cold front passage — rapid cooling

Miami (KMIA): - Sea breeze dominance: when south wind (160°–200°) with gusts ≥15 kt establishes, temperature caps at ~79°F - Sea breeze typically activates 1–3 PM EST - High humidity means the "feels like" temp differs from the station temp — market resolves on station temp, not heat index - Relatively low diurnal range (8–12°F) due to maritime influence

Seattle (KSEA): - Marine layer (W-NW wind, 250°–320°) causes consistent cooling that models overestimate - Pacific air masses create temperature ceilings that hold for

hours - Winter inversions can trap cold air in Puget Sound basin - Models frequently overshoot winter highs by 2–4°F

New York City (KLGA — LaGuardia): - Coastal effects from Long Island Sound create localized cooling/warming - Urban heat island effect strongest at night (raises lows more than highs) - W-NW wind brings marine influence similar to Seattle pattern - Wind channeling through Manhattan creates local effects at LaGuardia

Chicago (KORD — O'Hare): - Lake Michigan effect: E-NE winds in spring/summer cap temperatures below model projections - Runway heating effects in summer can produce station temps 1–2°F above surrounding area - In our data: 100% win rate (2/2 trades). Small sample, but Chicago patterns are the most predictable - Polar vortex events produce multi-day temperature locks with minimal diurnal variation

---

Part 4: The Gate Framework — Complete Rule Set

Philosophy: Why Hard Blocks Exist

Gates exist because certain weather patterns create failure modes that look like opportunities. The market scanner sees a 40% edge. The confidence model says 85%. The position sizer says $200. But there's a sea breeze coming that will cap the temperature 4°F below the forecast. If you take the trade, you lose.

Gates are the system's immune response. They check for known failure patterns and **hard-kill** the trade before the cusp model and position sizer waste your capital on a doomed thesis.

The gate philosophy: **it's always cheaper to skip a good trade than to take a bad one.** If a gate incorrectly blocks a winning trade, you miss a gain. If a gate correctly blocks a losing trade, you save your capital. Asymmetric — err on the side of blocking.

The Complete Gate Implementation

```
#!/usr/bin/env python3
"""
weather_gates.py - Trade-blocking gates for known weather failure modes.

Each gate checks for a specific meteorological pattern that invalidates
the standard trading thesis. Gates are checked BEFORE the cusp model
and position sizer - they are the first kill switch in the pipeline.

Gates return:
  blocked=True  → Trade is killed, no override possible
  blocked=False → Trade proceeds to cusp model
  caution=True  → Trade proceeds but with reduced position size

Usage:
```

```
    gates = WeatherGates('KMIA', 'Miami')
    result = gates.check_all_gates(
        scanner_pred_f=82.0,
        actual_5min_f=78.5,
        age_min=45,
        wind={'direction': 185, 'speed': 12, 'gust': 18}
    )

    if result['overall_blocked']:
        print(f"BLOCKED: {result['reason']}")
    else:
        # Proceed to cusp model
        pass
"""

from dataclasses import dataclass
from typing import Optional, Dict, Any
import json


@dataclass
class GateResult:
    """Result from a single gate check."""
    name: str
    blocked: bool
    caution: bool
    reason: str
    data: Optional[Dict] = None

    def to_dict(self):
        return {
            'name': self.name,
            'blocked': self.blocked,
            'caution': self.caution,
            'reason': self.reason,
            'data': self.data
        }


class WeatherGates:
    """
    Gate framework for blocking trades in known failure scenarios.

    Four gates, each targeting a specific meteorological pattern:
    1. Sea Breeze Cap - Miami: S wind + gusts = temp capped ~79°F
    2. Marine Layer - Seattle/NYC: W-NW wind = cooling cap
    3. Forecast Divergence - All cities: scanner vs actual >1.5°F for 30+ min
    4. Polar Vortex Lock - Chicago/NYC/Dallas: extreme cold, no diurnal range
```

```python
"""

# Station-to-city mapping
STATION_CITY = {
    'KATL': 'Atlanta',
    'KDAL': 'Dallas',
    'KMIA': 'Miami',
    'KSEA': 'Seattle',
    'KLGA': 'New York City',
    'KORD': 'Chicago'
}

def __init__(self, station: str, city: str = None):
    self.station = station.upper()
    self.city = city or self.STATION_CITY.get(self.station, 'Unknown')

def check_all_gates(
    self,
    scanner_pred_f: float = None,
    actual_5min_f: float = None,
    age_min: float = 0,
    wind: Dict = None,
    current_temp_f: float = None,
    seasonal_avg_f: float = None,
    trend_3hr: str = None
) -> Dict[str, Any]:
    """
    Run all applicable gates for this city.

    Returns:
        {
            'overall_blocked': bool,
            'overall_caution': bool,
            'reason': str,
            'gates': {gate_name: GateResult, ...},
            'position_modifier': float  # 1.0 = full, 0.5 = half, 0.0 = blocked
        }
    """
    gates = {}

    # Gate 1: Sea Breeze Cap (Miami only)
    gates['sea_breeze'] = self._check_sea_breeze(wind)

    # Gate 2: Marine Layer (Seattle, NYC)
    gates['marine_layer'] = self._check_marine_layer(wind)

    # Gate 3: Forecast Divergence (all cities)
    gates['forecast_divergence'] = self._check_forecast_divergence(
```

```
        scanner_pred_f, actual_5min_f, age_min
    )

    # Gate 4: Polar Vortex Lock (Chicago, NYC, Dallas in winter)
    gates['polar_vortex'] = self._check_polar_vortex(
        current_temp_f, seasonal_avg_f, trend_3hr
    )

    # Aggregate results
    any_blocked = any(g.blocked for g in gates.values())
    any_caution = any(g.caution for g in gates.values())

    # Position modifier: 1.0 (full), 0.5 (caution), 0.0 (blocked)
    if any_blocked:
        modifier = 0.0
        reason = '; '.join(
            g.reason for g in gates.values() if g.blocked
        )
    elif any_caution:
        modifier = 0.5
        reason = '; '.join(
            g.reason for g in gates.values() if g.caution
        )
    else:
        modifier = 1.0
        reason = 'All gates clear'

    return {
        'overall_blocked': any_blocked,
        'overall_caution': any_caution,
        'reason': reason,
        'gates': {name: g.to_dict() for name, g in gates.items()},
        'position_modifier': modifier
    }

#   Gate 1: Sea Breeze Cap

def _check_sea_breeze(self, wind: Dict = None) -> GateResult:
    """
    Miami: South wind (160°-200°) with gusts  15 kt = sea breeze
    circulation. Temperature caps at ~79°F regardless of forecast.

    This gate has NEVER been wrong in our data. When the wind
    signature matches, the cap holds. Period.

    Threshold:
      - Wind direction: 160°-200° (south)
      - Gust speed:  15 knots
```

```
  - Blocks: Any trade expecting temp > 79°F
"""
if self.city != 'Miami':
    return GateResult(
        name='sea_breeze', blocked=False, caution=False,
        reason='N/A (not Miami)'
    )

if not wind:
    return GateResult(
        name='sea_breeze', blocked=False, caution=False,
        reason='No wind data available'
    )

direction = wind.get('direction', 0)
gust = wind.get('gust', 0)
speed = wind.get('speed', 0)

is_south_wind = 160 <= direction <= 200
has_significant_gust = gust and gust >= 15

if is_south_wind and has_significant_gust:
    return GateResult(
        name='sea_breeze',
        blocked=True,
        caution=False,
        reason=(
            f"Sea breeze ACTIVE - S wind {direction}°, "
            f"gust {gust}kt. Temp capped ~79°F. "
            f"Block all trades expecting >79°F."
        ),
        data={
            'wind_dir': direction,
            'gust_kt': gust,
            'temp_cap_f': 79,
            'confidence': 'HIGH'
        }
    )

# Partial signature: south wind but no strong gust yet
if is_south_wind and speed >= 10:
    return GateResult(
        name='sea_breeze',
        blocked=False,
        caution=True,
        reason=(
            f"Sea breeze FORMING - S wind {direction}° at {speed}kt, "
            f"no strong gusts yet. Monitor closely. "
```

```
                f"Reduce position 50% on 80°F+ trades."
            ),
            data={
                'wind_dir': direction,
                'speed_kt': speed,
                'status': 'forming'
            }
        )

    return GateResult(
        name='sea_breeze', blocked=False, caution=False,
        reason='Sea breeze not active (wind pattern doesn\'t match)'
    )

#   Gate 2: Marine Layer

def _check_marine_layer(self, wind: Dict = None) -> GateResult:
    """
    Seattle & NYC: West-northwest wind (250°-320°) pushes marine air
    inland, creating a cooling cap that forecast models consistently
    overestimate - especially in winter.

    This gate raises CAUTION (not hard block) because marine layer
    effects are less deterministic than sea breeze. Action: reduce
    position by 50%, require wider safety margin.

    Threshold:
      - Wind direction: 250°-320° (W-NW)
      - Speed:  8 knots (lighter winds can still carry marine air)
      - Effect: Models overestimate high by 2-4°F
    """
    if self.city not in ['Seattle', 'New York City']:
        return GateResult(
            name='marine_layer', blocked=False, caution=False,
            reason=f'N/A (not Seattle/NYC)'
        )

    if not wind:
        return GateResult(
            name='marine_layer', blocked=False, caution=False,
            reason='No wind data available'
        )

    direction = wind.get('direction', 0)
    speed = wind.get('speed', 0)

    is_west_nw = 250 <= direction <= 320
    has_speed = speed >= 8
```

```python
    if is_west_nw and has_speed:
        return GateResult(
            name='marine_layer',
            blocked=False,
            caution=True,
            reason=(
                f"Marine layer ACTIVE - W-NW wind {direction}° at "
                f"{speed}kt. Expect cooling. Models may overestimate "
                f"high by 2-4°F. Reduce position 50%, widen margin."
            ),
            data={
                'wind_dir': direction,
                'speed_kt': speed,
                'expected_model_overshoot_f': '2-4',
                'position_modifier': 0.5
            }
        )

    return GateResult(
        name='marine_layer', blocked=False, caution=False,
        reason='Marine layer not active (wind pattern doesn\'t match)'
    )

#   Gate 3: Forecast Divergence

def _check_forecast_divergence(
    self,
    scanner_pred_f: float = None,
    actual_5min_f: float = None,
    age_min: float = 0
) -> GateResult:
    """
    All cities: When the scanner prediction (based on NWS forecast)
    diverges from actual 5-minute observations by >1.5°F for more
    than 30 minutes, the forecast is wrong. Don't trade on wrong
    forecasts.

    This catches the failure mode where you trust stale forecast data.
    By 2 PM, if the temperature hasn't reached where the morning
    forecast said it would be, the forecast is busted.

    Thresholds:
      - Divergence: >1.5°F between scanner prediction and actual
      - Duration: >30 minutes since scanner prediction was generated
      - Action: Hard block - do not enter new positions
    """
    if scanner_pred_f is None or actual_5min_f is None:
```

```
    return GateResult(
        name='forecast_divergence', blocked=False, caution=False,
        reason='Missing scanner or actual temperature data'
    )

divergence = abs(scanner_pred_f - actual_5min_f)

if divergence > 1.5 and age_min > 30:
    return GateResult(
        name='forecast_divergence',
        blocked=True,
        caution=False,
        reason=(
            f"Forecast DIVERGENCE - Scanner predicted "
            f"{scanner_pred_f:.1f}°F, actual is {actual_5min_f:.1f}°F "
            f"(diff {divergence:.1f}°F, {age_min:.0f} min old). "
            f"Forecast wrong. Block new entries."
        ),
        data={
            'scanner_pred_f': scanner_pred_f,
            'actual_f': actual_5min_f,
            'divergence_f': round(divergence, 1),
            'age_min': age_min
        }
    )

if divergence > 1.0 and age_min > 20:
    return GateResult(
        name='forecast_divergence',
        blocked=False,
        caution=True,
        reason=(
            f"Forecast drifting - Scanner predicted "
            f"{scanner_pred_f:.1f}°F, actual {actual_5min_f:.1f}°F "
            f"(diff {divergence:.1f}°F, {age_min:.0f} min). "
            f"Approaching divergence threshold. Reduce confidence."
        ),
        data={
            'scanner_pred_f': scanner_pred_f,
            'actual_f': actual_5min_f,
            'divergence_f': round(divergence, 1),
            'age_min': age_min,
            'position_modifier': 0.75
        }
    )

return GateResult(
    name='forecast_divergence', blocked=False, caution=False,
```

```
        reason='Forecast and actual temperatures aligned'
    )

#   Gate 4: Polar Vortex Lock

def _check_polar_vortex(
    self,
    current_temp_f: float = None,
    seasonal_avg_f: float = None,
    trend_3hr: str = None
) -> GateResult:
    """
    Chicago, NYC, Dallas (winter): When a polar vortex event locks in,
    temperatures can be 15-25°F below seasonal norms with near-zero
    diurnal variation. The daily high might be -5°F. Standard brackets
    are all NO. The only tradeable brackets are extreme lows with
    thin liquidity.

    Thresholds:
      - Current temp: >20°F below seasonal average
      - Trend: No warming over past 3 hours
      - Action: Hard block (extreme cold = untradeable)
    """
    eligible_cities = ['Chicago', 'New York City', 'Dallas']
    if self.city not in eligible_cities:
        return GateResult(
            name='polar_vortex', blocked=False, caution=False,
            reason=f'N/A (not applicable to {self.city})'
        )

    if current_temp_f is None or seasonal_avg_f is None:
        return GateResult(
            name='polar_vortex', blocked=False, caution=False,
            reason='Missing temperature or seasonal data'
        )

    departure = seasonal_avg_f - current_temp_f

    if departure > 20 and trend_3hr in ['flat', 'cooling', None]:
        return GateResult(
            name='polar_vortex',
            blocked=True,
            caution=False,
            reason=(
                f"Polar vortex LOCK - Current {current_temp_f:.0f}°F is "
                f"{departure:.0f}°F below seasonal avg ({seasonal_avg_f:.0f}°F). "
                f"No warming trend. Standard brackets untradeable."
            ),
```

```
                data={
                    'current_f': current_temp_f,
                    'seasonal_avg_f': seasonal_avg_f,
                    'departure_f': round(departure, 1),
                    'trend': trend_3hr
                }
            )

        if departure > 15:
            return GateResult(
                name='polar_vortex',
                blocked=False,
                caution=True,
                reason=(
                    f"Cold outbreak - {current_temp_f:.0f}°F is "
                    f"{departure:.0f}°F below seasonal avg. "
                    f"Elevated uncertainty on bracket resolution. "
                    f"Reduce position size."
                ),
                data={
                    'departure_f': round(departure, 1),
                    'position_modifier': 0.5
                }
            )

        return GateResult(
            name='polar_vortex', blocked=False, caution=False,
            reason='Temperature within normal range for season'
        )


#   Running All Gates: Example

if __name__ == '__main__':
    # Example 1: Miami with sea breeze
    print("=" * 60)
    print("Example 1: Miami with active sea breeze")
    print("=" * 60)

    gates = WeatherGates('KMIA', 'Miami')
    result = gates.check_all_gates(
        scanner_pred_f=83.0,
        actual_5min_f=78.2,
        age_min=45,
        wind={'direction': 185, 'speed': 14, 'gust': 22}
    )

    print(f"Overall: {'  BLOCKED' if result['overall_blocked'] else '  CLEAR'}")
```

```python
    print(f"Reason: {result['reason']}")
    print(f"Position modifier: {result['position_modifier']}")
    for name, gate in result['gates'].items():
        icon = ' ' if gate['blocked'] else (' ' if gate['caution'] else ' ')
        print(f"  {icon} {name}: {gate['reason']}")

    # Example 2: Atlanta, clear conditions
    print(f"\n{'=' * 60}")
    print("Example 2: Atlanta, all clear")
    print("=" * 60)

    gates = WeatherGates('KATL', 'Atlanta')
    result = gates.check_all_gates(
        scanner_pred_f=65.0,
        actual_5min_f=63.8,
        age_min=20,
        wind={'direction': 220, 'speed': 8, 'gust': None}
    )

    print(f"Overall: {'  BLOCKED' if result['overall_blocked'] else '  CLEAR'}")
    print(f"Position modifier: {result['position_modifier']}")
    for name, gate in result['gates'].items():
        icon = ' ' if gate['blocked'] else (' ' if gate['caution'] else ' ')
        print(f"  {icon} {name}: {gate['reason']}")
```

Expected output:

```
============================================================
Example 1: Miami with active sea breeze
============================================================
Overall:   BLOCKED
Reason: Sea breeze ACTIVE - S wind 185°, gust 22kt. Temp capped ~79°F. Block all trades expecti
Position modifier: 0.0
   sea_breeze: Sea breeze ACTIVE - S wind 185°, gust 22kt. Temp capped ~79°F. Block all trades
   marine_layer: N/A (not Seattle/NYC)
   forecast_divergence: Forecast DIVERGENCE - Scanner predicted 83.0°F, actual is 78.2°F (diff
   polar_vortex: N/A (not applicable to Miami)

============================================================
Example 2: Atlanta, all clear
============================================================
Overall:   CLEAR
Position modifier: 1.0
   sea_breeze: N/A (not Miami)
   marine_layer: N/A (not Seattle/NYC)
   forecast_divergence: Forecast and actual temperatures aligned
   polar_vortex: N/A (not applicable to Atlanta)
```

Gate Override Policy

Sea breeze: Never override. The pattern has held every time in our data. The only theoretical exception — a synoptic offshore wind strong enough to suppress the sea breeze — doesn't happen when you're already seeing gusting south winds.

Marine layer: Overridable with evidence. If METAR shows temperature climbing despite W-NW wind, the marine layer may be shallow or breaking up. You can proceed with reduced position (the gate already sets 0.5 modifier).

Forecast divergence: Override only if you understand why the forecast was wrong and have a revised thesis. Example: NWS forecast said 78°F but the morning was cloudy. At 1 PM, clouds break and temperature starts climbing 3°F/hr. The forecast was wrong about timing but might be right about magnitude. You can trade the revised thesis — but size conservatively.

Polar vortex: Never override. Extreme cold events are untradeable. Liquidity is thin, brackets are meaningless, and the temperature pattern is locked.

Part 5: The Cusp Model — Full Mathematical Treatment

The Core Insight: Rounding Creates Discontinuities

Weather Underground displays temperatures as whole-degree Fahrenheit. Standard rounding applies:

```
Raw °F  X.5  →  displays as (X+1)°F
Raw °F < X.5  →  displays as X°F
```

This creates a discontinuous function. At 57.49°F, WU displays 57. At 57.50°F, WU displays 58. A **0.01°F change** — immeasurably small — flips the displayed value by a full degree. And that flip can mean the difference between a bracket resolving YES or NO.

The cusp model exists to detect when the actual temperature is near these rounding boundaries and to quantify the exact risk.

The Mathematics of WU Rounding

For a bracket defined as `[L, U]°F` (e.g., 56–57°F), the actual temperature ranges that resolve into this bracket are:

```
Lower boundary:  L - 0.5     actual_f  <  L      → rounds to L (in bracket)
Core range:      L           actual_f     U       → in bracket
Upper boundary:  U        <  actual_f  <  U + 0.5  → rounds to U (in bracket)
```

The full range where WU displays a value within [L, U]:

```
actual_f  [(L - 0.5), (U + 0.5))
```

For the 56–57°F bracket:

```
actual_f   [55.5, 57.5)

55.49°F → WU: 55 → OUT (below bracket)
55.50°F → WU: 56 → IN  (lower edge)
57.00°F → WU: 57 → IN  (mid-bracket)
57.49°F → WU: 57 → IN  (upper edge)
57.50°F → WU: 58 → OUT (above bracket)
```

Safety Margin: The Distance That Matters

The **safety margin** is the distance from the current temperature to the nearest WU rounding boundary that would move the display out of (or into) your bracket.

For a YES bet on the 56–57°F bracket:

```python
def calculate_safety_margin(actual_f: float, bracket_lower: int, bracket_upper: int,
                            direction: str = 'YES') -> dict:
    """
    Calculate the WU rounding safety margin.

    This is THE critical calculation. The Feb 15 disaster happened because
    we calculated the raw distance to 58°F (0.98°F) instead of the distance
    to the WU rounding threshold of 57.5°F (0.48°F).

    For YES bets: your risk is the temperature moving OUT of the bracket
      - Upper risk: actual_f crossing (bracket_upper + 0.5) → rounds to upper+1
      - Lower risk: actual_f crossing (bracket_lower - 0.5) → rounds to lower-1

    Returns the MINIMUM of upper and lower margins (worst case).
    """
    # WU rounding boundaries
    upper_boundary = bracket_upper + 0.5   # Cross this → rounds above bracket
    lower_boundary = bracket_lower - 0.5   # Cross this → rounds below bracket

    # WU display of current temp
    wu_display = round(actual_f)
    in_bracket = bracket_lower <= wu_display <= bracket_upper

    if direction == 'YES':
        if in_bracket:
            # Distance to nearest exit
            margin_up = upper_boundary - actual_f    # How far to rounding out (top)
            margin_down = actual_f - lower_boundary  # How far to rounding out (bottom)
            effective_margin = min(margin_up, margin_down)

            # Identify which side is the risk
            risk_side = 'upper' if margin_up < margin_down else 'lower'
        else:
            # Not in bracket - need temperature to move INTO bracket
```

```
        if wu_display < bracket_lower:
            effective_margin = -(lower_boundary - actual_f)  # Negative = needs to move
        else:
            effective_margin = -(actual_f - upper_boundary)
        risk_side = 'outside'
else:  # NO bet
    if not in_bracket:
        # Good - we want it to stay out
        if wu_display < bracket_lower:
            effective_margin = lower_boundary - actual_f
        else:
            effective_margin = actual_f - upper_boundary
        risk_side = 'outside_safe'
    else:
        # Bad - it's in the bracket and we bet NO
        effective_margin = -min(
            upper_boundary - actual_f,
            actual_f - lower_boundary
        )
        risk_side = 'inside_danger'

# Risk assessment
abs_margin = abs(effective_margin)
if abs_margin >= 1.5:
    risk_level = 'LOW'
elif abs_margin >= 1.0:
    risk_level = 'LOW'
elif abs_margin >= 0.5:
    risk_level = 'MEDIUM'
elif abs_margin >= 0.3:
    risk_level = 'HIGH'
else:
    risk_level = 'EXTREME'

# Position allowed?
trade_allowed = effective_margin >= 0.3  # Hard floor

return {
    'actual_f': actual_f,
    'wu_display': wu_display,
    'in_bracket': in_bracket,
    'bracket': f"{bracket_lower}-{bracket_upper}°F",
    'upper_boundary': upper_boundary,
    'lower_boundary': lower_boundary,
    'effective_margin': round(effective_margin, 2),
    'risk_side': risk_side,
    'risk_level': risk_level,
    'trade_allowed': trade_allowed,
```

```
    }


#   The Feb 15 Scenario: Why This Calculation Saves $300

feb15 = calculate_safety_margin(
    actual_f=57.02,
    bracket_lower=56,
    bracket_upper=57,
    direction='YES'
)

print("Feb 15 Analysis (the $300 disaster scenario):")
print(f"  Actual temp:      {feb15['actual_f']}°F")
print(f"  WU displays:      {feb15['wu_display']}°F")
print(f"  In bracket:       {feb15['in_bracket']}")
print(f"  Safety margin:    {feb15['effective_margin']}°F")
print(f"  Risk level:       {feb15['risk_level']}")
print(f"  Trade allowed:    {feb15['trade_allowed']}")
print()
print(f"  Raw distance to 58°F:  {58 - 57.02:.2f}°F  ← What we calculated")
print(f"  WU rounding margin:    {feb15['effective_margin']:.2f}°F  ← What we should have calcu
print(f"  Difference:            {0.98 - 0.48:.2f}°F of false security")
```

Output:

```
Feb 15 Analysis (the $300 disaster scenario):
  Actual temp:      57.02°F
  WU displays:      57°F
  In bracket:       True
  Safety margin:    0.48°F
  Risk level:       HIGH
  Trade allowed:    True (but only quarter position!)

  Raw distance to 58°F:  0.98°F  ← What we calculated
  WU rounding margin:    0.48°F  ← What we should have calculated
  Difference:            0.50°F of false security
```

At 0.48°F margin, the position sizer caps at 25% of maximum. On a $300 max position, that's $75. The loss would have been $75 instead of $300. Painful but survivable.

Oscillation Detection: The Hidden Cusp Signal

When the real temperature is sitting right on a Celsius boundary (e.g., 14.5°C), something distinctive happens in the 5-minute whole-degree observations: they oscillate.

```
Time    5-min obs (°C)    Actual (estimated)

:00     14                ~14.3°C
```

```
:05     15                ~14.6°C
:10     14                ~14.4°C
:15     15                ~14.5°C
:20     14                ~14.4°C
:25     15                ~14.6°C
```

The 5-minute observations round to whole Celsius. When the actual temperature oscillates around 14.5°C, you see the readings flip between 14 and 15. Most people see this and think "sensor noise." We see it and think "cusp at 14.5°C = 58.1°F."

The oscillation detector:

```python
def detect_oscillation(readings: list, min_flips: int = 3) -> dict:
    """
    Detect temperature oscillation in 5-minute whole-degree readings.

    Oscillation between adjacent values (e.g., 14°C  15°C) indicates
    the actual temperature is near the boundary between them.

    Args:
        readings: List of whole-degree Celsius observations (most recent last)
        min_flips: Minimum number of value changes to confirm oscillation

    Returns:
        {
            'oscillating': bool,
            'cusp_c': float or None,      # Estimated cusp temperature
            'cusp_f': float or None,      # Cusp in Fahrenheit
            'wu_display': int or None,    # What WU would show at the cusp
            'confidence': float,           # 0.0 to 1.0
            'flip_count': int,
            'values_seen': set
        }
    """
    if len(readings) < 4:
        return {
            'oscillating': False, 'cusp_c': None, 'cusp_f': None,
            'wu_display': None, 'confidence': 0.0,
            'flip_count': 0, 'values_seen': set()
        }

    # Count value changes (flips)
    flips = 0
    values = set()
    for i in range(1, len(readings)):
        if readings[i] != readings[i-1]:
            flips += 1
        values.add(readings[i])
    values.add(readings[0])
```

```python
    # Oscillation requires:
    # 1. Exactly 2 adjacent values
    # 2. Multiple flips between them
    is_oscillating = (
        len(values) == 2 and
        flips >= min_flips and
        abs(max(values) - min(values)) == 1  # Adjacent whole degrees
    )

    if not is_oscillating:
        return {
            'oscillating': False, 'cusp_c': None, 'cusp_f': None,
            'wu_display': None, 'confidence': 0.0,
            'flip_count': flips, 'values_seen': values
        }

    # Estimate the cusp temperature
    low_val = min(values)
    high_val = max(values)
    cusp_c = (low_val + high_val) / 2.0  # Midpoint = boundary

    # Refine: if more readings show the high value, actual is slightly above mid
    high_count = sum(1 for r in readings if r == high_val)
    total = len(readings)
    bias = (high_count / total - 0.5) * 0.4  # ±0.2°C adjustment
    cusp_c_refined = cusp_c + bias

    cusp_f = cusp_c_refined * 9.0 / 5.0 + 32.0
    wu_display = round(cusp_f)

    # Confidence based on flip count and reading count
    confidence = min(1.0, flips / (len(readings) * 0.8))

    return {
        'oscillating': True,
        'cusp_c': round(cusp_c_refined, 1),
        'cusp_f': round(cusp_f, 1),
        'wu_display': wu_display,
        'confidence': round(confidence, 2),
        'flip_count': flips,
        'values_seen': values
    }


# Example from real KATL data:
readings = [14, 15, 14, 14, 15, 14, 15]
result = detect_oscillation(readings)
```

```
print(f"Oscillating: {result['oscillating']}")
print(f"Estimated cusp: {result['cusp_c']}°C = {result['cusp_f']}°F")
print(f"WU would display: {result['wu_display']}°F")
print(f"Confidence: {result['confidence']}")
print(f"Flip count: {result['flip_count']}")
```

The Full Cusp Model Pipeline

The cusp model combines T-group parsing, oscillation detection, trend analysis, and WU rounding into a single estimation pipeline:

```
class METARCuspModel:
    """
    Full cusp model for estimating current temperature with WU-precision.

    Combines:
    1. Latest METAR T-group (0.1°C precision)
    2. 5-minute observation oscillation detection
    3. Temperature trend analysis (warming/cooling/stalling)
    4. WU rounding safety margin calculation

    The model outputs:
    - Best estimate of current temperature (°F, 0.01 resolution)
    - What WU is currently displaying
    - Safety margin to nearest rounding boundary
    - Trend direction and rate
    - Action recommendation (HOLD, EXIT, ENTRY_OK, SKIP)
    """

    def __init__(self, station: str):
        self.station = station.upper()
        self._last_metar = None
        self._5min_history = []

    def estimate_current_temp(
        self,
        metar_tgroup: str = None,
        metar_age_min: float = None,
        five_min_readings_c: list = None,
        five_min_latest_c: float = None
    ) -> dict:
        """
        Estimate current temperature using best available data.

        Priority:
        1. If METAR T-group is <10 min old → use directly (highest precision)
        2. If T-group is 10-50 min old → combine with 5-min trend
        3. If T-group is >50 min old → STALE, use 5-min only (lower confidence)
```

```
    Returns comprehensive estimation dict.
    """
    result = {
        'status': 'OK',
        'station': self.station,
    }

    # Parse T-group if available
    if metar_tgroup:
        tg = parse_tgroup(metar_tgroup)
        if tg:
            result['last_metar_tc'] = tg['temp_c']
            result['last_metar_tf'] = tg['temp_f']
            result['last_metar_wu'] = tg['wu_display']
            result['last_metar_age_min'] = metar_age_min or 0

    # Check 5-min oscillation
    if five_min_readings_c and len(five_min_readings_c) >= 4:
        osc = detect_oscillation(five_min_readings_c)
        result['oscillation'] = osc
    else:
        result['oscillation'] = {'oscillating': False}

    # Determine best estimate
    if metar_tgroup and metar_age_min and metar_age_min < 10:
        # Fresh METAR - use T-group directly
        result['estimated_tc'] = tg['temp_c']
        result['estimated_tf'] = tg['temp_f']
        result['estimated_wu'] = tg['wu_display']
        result['confidence'] = 'HIGH'
        result['source'] = 'metar_tgroup'

    elif metar_tgroup and metar_age_min and metar_age_min < 50:
        # Aging METAR - combine with 5-min trend
        if five_min_latest_c is not None:
            # Use T-group as baseline, 5-min as trend indicator
            delta_c = five_min_latest_c - round(tg['temp_c'])
            adjusted_tc = tg['temp_c'] + (delta_c * 0.3)  # Partial adjustment
            adjusted_tf = adjusted_tc * 9.0 / 5.0 + 32.0
            result['estimated_tc'] = round(adjusted_tc, 1)
            result['estimated_tf'] = round(adjusted_tf, 2)
            result['estimated_wu'] = round(adjusted_tf)
            result['confidence'] = 'MEDIUM'
            result['source'] = 'metar_tgroup + 5min_trend'
            result['delta_tc'] = round(delta_c, 1)
        else:
            result['estimated_tc'] = tg['temp_c']
```

```
            result['estimated_tf'] = tg['temp_f']
            result['estimated_wu'] = tg['wu_display']
            result['confidence'] = 'MEDIUM'
            result['source'] = 'metar_tgroup (aging)'

    elif five_min_latest_c is not None:
        # No recent METAR - 5-min only (lower precision)
        est_tf = five_min_latest_c * 9.0 / 5.0 + 32.0
        result['estimated_tc'] = five_min_latest_c
        result['estimated_tf'] = round(est_tf, 2)
        result['estimated_wu'] = round(est_tf)
        result['confidence'] = 'LOW'
        result['source'] = '5min_only'

    else:
        result['status'] = 'NO_DATA'
        result['confidence'] = 'NONE'
        return result

    # Trend analysis
    if five_min_readings_c and len(five_min_readings_c) >= 3:
        recent = five_min_readings_c[-3:]
        if all(recent[i] <= recent[i+1] for i in range(len(recent)-1)):
            result['trend'] = 'WARMING'
        elif all(recent[i] >= recent[i+1] for i in range(len(recent)-1)):
            result['trend'] = 'COOLING'
        elif max(recent) - min(recent) <= 1:
            result['trend'] = 'STALLING'
        else:
            result['trend'] = 'VARIABLE'

        # Rate of change (°C/hr, approximated from 5-min intervals)
        if len(five_min_readings_c) >= 2:
            rate_per_5min = five_min_readings_c[-1] - five_min_readings_c[-2]
            rate_per_hr = rate_per_5min * 12
            result['rate_c_per_hr'] = round(rate_per_hr, 1)
    else:
        result['trend'] = 'UNKNOWN'

    # Generate action recommendation
    result['action'] = self._recommend_action(result)

    return result

def _recommend_action(self, estimation: dict) -> str:
    """Generate action recommendation based on estimation."""
    confidence = estimation.get('confidence', 'NONE')
    trend = estimation.get('trend', 'UNKNOWN')
```

```
if confidence == 'NONE' or estimation.get('status') != 'OK':
    return 'NO_DATA'

if confidence == 'LOW':
    return 'WAIT_FOR_METAR'

if trend == 'STALLING':
    return 'HOLD'  # Temperature appears to have peaked
elif trend == 'WARMING':
    return 'MONITOR'  # Still climbing - wait for peak
elif trend == 'COOLING':
    return 'HOLD'  # Past peak, high is locked
else:
    return 'MONITOR'
```

Cusp Model Trading Scenarios

Scenario 1: Temperature locked in cusp zone — market underpricing the bracket

Temperature has been oscillating between 14°C and 15°C for 45 minutes. Cusp model estimates 14.5°C = 58.1°F. WU displays 58°F. The 58–59°F bracket is priced at 35 cents because traders see the NWS forecast calling for 61°F and expect the temperature to blow through.

But the oscillation pattern plus a stalling trend suggests the peak is here. The temperature has been oscillating — not climbing — for 45 minutes. If this is the peak, the high is locked at 58°F. The 58–59°F bracket should be priced at 65 cents.

Edge: 30 cents. Action: BUY YES at 35 cents.

Scenario 2: Temperature approaching boundary from below — market overpricing

Current temp: 69.3°F. The ≥70°F bracket is priced at 72 cents because NWS forecasts 71°F. But the trend shows warming has stalled for 20+ minutes. The cusp model calculates 0.7°F margin to the 69.5°F WU rounding boundary — the temperature needs to rise another 0.2°F in the actual reading to round to 70°F. With a stalling trend, this is unlikely.

Edge: selling overpriced YES contracts (or buying NO).

Scenario 3: Oscillation reveals hidden risk on an existing position

You hold the 64–65°F bracket. Latest T-group shows 65.1°F (margin = 0.39°F to upper boundary). But the 5-minute readings are oscillating: 18, 19, 18, 19, 18. The cusp model detects oscillation at 18.5°C = 65.3°F. Actual might be higher than the T-group suggested. New margin estimate: 0.2°F.

Action: EXIT — margin is below the 0.3°F floor.

Part 6: OpenClaw Setup — Advanced Configuration

Prerequisites (From the Starter Guide)

You already have: - OpenClaw installed (`npm install -g openclaw`) - A Telegram bot connected - Basic agent workspace set up - Gateway running (`openclaw gateway start`)

This section covers the **advanced configuration** that turns your basic agent into a trading operation.

Agent Workspace Structure

Your trading agent workspace should look like this:

```
~/trading-analyst/agents/trading-bot/
   IDENTITY.md          # Agent identity and mission
   SOUL.md              # Voice and personality
   RULES.md             # Trading rules (non-negotiable)
   HEARTBEAT.md         # Scheduled check sequences
   MEMORY.md            # Learned patterns and lessons
   AGENTS.md            # Workspace configuration

   memory/              # Daily notes (auto-generated)
       2026-03-08.md
       2026-03-09.md

   ~/life/              # Knowledge graph (PARA structure)
       projects/
           polymarket-trading/
               summary.md
               items.json
       areas/
           weather-markets/
       resources/
           metar-reference/
```

IDENTITY.md — Trading Agent

```
# IDENTITY.md - Trading Agent Identity

- Name: Sentinel (or your preferred name)
- Role: AI Weather Trading Analyst
- Mission: Identify and exploit mispricings in Polymarket weather markets
  with disciplined risk management
- Emoji:

## Operating Mode
This agent is a specialized trading analyst, not a general assistant.
Every heartbeat, every research task, every alert is filtered through:
```

```
does this help us make better trades?

## Focus Areas
1. **Real-time monitoring** - METAR data, Wethr.net push, temperature trends
2. **Opportunity detection** - Market scanner, cusp model, edge calculation
3. **Risk management** - Gate checks, safety margins, position sizing
4. **Performance tracking** - Decision logging, pattern extraction, P&L

## Core Principle
Position sizing is more important than being right about the weather.
We can be wrong 40% of the time and still be profitable if we size correctly.
We cannot be right 80% of the time and survive if we size based on emotion.
```

RULES.md — Non-Negotiable Trading Rules

```
# RULES.md - Trading Rules

These rules are NON-NEGOTIABLE. The agent cannot override them.
They exist because we lost $300 learning what happens without them.

## Position Sizing (The Most Important Section)

| Safety Margin | Max Position | Risk Level |
|---------------|-------------|------------|
| > 1.5°F | 100% of max | LOW   |
| > 1.0°F | 100% of max | LOW   |
| 0.5-1.0°F | 50% of max | MEDIUM   |
| 0.3-0.5°F | 25% of max | HIGH   |
| < 0.3°F | $0 - NO TRADE | EXTREME   |

**The 0.3°F floor is absolute.** No override by confidence, edge size,
or any other factor.

**Maximum single position:** $300 (hard cap regardless of bankroll)
**Maximum daily risk:** $200 total across all positions

## Entry Requirements (ALL must pass)
- [ ] Minimum 2 confirming data sources
- [ ] All weather gates clear (or caution with adjusted size)
- [ ] WU rounding safety margin calculated and logged
- [ ] Position size from calculator (not from intuition)
- [ ] Entry logged to decisions.db before execution

## Exit Rules
- EXIT immediately if safety margin drops below 0.2°F
- EXIT if any weather gate triggers after entry
- EXIT if 3 consecutive METAR readings show adverse trend
- NEVER hold past resolution hoping for reversal
```

```
## Circuit Breakers
- 3 consecutive losses → pause 24 hours
- Single loss > $100 → mandatory review before next trade
- Win rate below 50% over rolling 20 trades → pause trading
- Maximum 4 trades per day

## Time-of-Day Rules
- After 3 PM local: cap confidence at 90%
- After 4 PM local: reduce position by 30%
- No new entries after 5 PM unless margin > 1.5°F

## Data Freshness
- Never trade on METAR older than 50 minutes
- Never trade when METAR and Wethr.net disagree by > 2°F
- Never trade on a suspect/rejected temperature reading
```

HEARTBEAT.md — Automated Trading Schedule

```
# HEARTBEAT.md - Scheduled Checks

## Market Hours Heartbeat (Every 30 Minutes, 10 AM - 9 PM EST)

### Sequence
1. Fetch current METAR for all 6 stations
2. Update Wethr.net WU highs
3. For each open position:
   a. Recalculate cusp model safety margin
   b. Run all weather gates
   c. If margin < 0.3°F → EXIT ALERT (Telegram, urgent)
   d. If margin < 0.5°F → CAUTION ALERT
   e. If position flipped out of bracket → LOSS ALERT
4. Scan for new opportunities:
   a. Run `python3 best_trades.py` in trading system directory
   b. If edge > 15% and all gates clear → OPPORTUNITY ALERT
5. Log heartbeat results to daily notes

### Alert Format
```

□ OPPORTUNITY: {city} {bracket} Edge: {edge}% | Market: ${price} | Fair: ${fair_value} Margin: {margin}°F | Gates: {status} Recommended: ${position} ({risk_level})

```
## Morning Briefing (8:00 AM EST)
- NWS forecast highs for all 6 cities
- Wethr.net overnight WU high/low
- Active weather patterns (fronts, sea breeze risk, inversions)
- Top 2-3 opportunities with sizing recommendations
- Open position status from previous day
```

```
## Evening Review (9:00 PM EST)
- Resolution status for all day's markets
- P&L for resolved positions
- Rolling 20-trade win rate update
- Positions carrying overnight (if any)
```

MEMORY.md — Learned Patterns

```
# MEMORY.md - Trading Patterns & Lessons

## The Feb 15 Rule (Permanent)
Never forget: 0.48°F margin + $300 position = disaster.
Always calculate WU rounding margin, not raw temperature distance.

## City Patterns
- **Atlanta:** Late afternoon warming possible after rain stops (cloud breaks)
- **Miami:** Sea breeze cap at ~79°F when S wind + gusts (never override)
- **Seattle:** Marine layer with W-NW wind - models overshoot by 2-4°F
- **Dallas:** KDAL  KDFW. Always verify station. 30°F+ diurnal swings in spring.
- **Chicago:** Most predictable patterns. 100% win rate in our data.
- **NYC:** Coastal effects, harbor influence on LaGuardia readings

## Winning Trade Profile (From Feb Audit)
All 4 February winners shared:
1. Entry before noon
2. Held to resolution (no panic selling)
3. METAR confirmation within 30 min of entry
4. 2+ confirming data sources

## Losing Trade Profile
11 February losers had one or more of:
- Late entry (after 3 PM)
- Thin safety margin (< 0.5°F)
- Single data source
- Panic interpretation of one METAR reading
- Position too large for available margin

## The Confidence Paradox
"High confidence" trades had 25% win rate and -$54 P&L.
"Medium confidence" trades had 25% win rate and +$21 P&L.
→ High confidence leads to bigger sizing, which amplifies losses.
→ Cap conviction. Let the margin determine the position, not the feeling.

## Signal Reliability
- safe_bullish: 100% win rate (1/1), +$75 - FOLLOW
- locked_win_metar: 50% (1/2), -$12 - PROCEED WITH CAUTION
- BUY_NO: 33% (1/3), -$17 - ONLY with strong margin
```

```
- manual_high_edge: 0% (0/2), -$42 - AVOID
- low_risk_contrarian: 0% (0/2), -$44 - AVOID
```

Advanced OpenClaw Configuration

For the trading agent, configure the gateway with trading-specific settings:

```
# Set the model for trading analysis (Sonnet for decisions, Haiku for monitoring)
openclaw config set llm.model claude-sonnet-4-6

# Configure heartbeat interval (30 minutes during market hours)
openclaw config set agent.heartbeatInterval 1800000

# Set timezone for market hours
openclaw config set agent.timezone "America/New_York"
```

The agent will use its HEARTBEAT.md to determine what to check during each heartbeat. During market hours (10 AM – 9 PM EST), every 30-minute heartbeat triggers the full trading pipeline: METAR fetch → gates → cusp model → scanner → alerts.

Part 7: The Python Trading Bot — Installation & Setup

Prerequisites

- Python 3.10+ (verify: `python3 --version`)
- pip (verify: `pip3 --version`)
- Git (verify: `git --version`)
- A terminal: macOS Terminal, Linux shell, or WSL on Windows
- OpenClaw running from the Starter Guide setup

Installation

```
# 1. Clone the trading system repository
cd ~/
git clone https://github.com/YOUR_REPO/polymarket-trading-bot.git
cd polymarket-trading-bot

# 2. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate  # Linux/macOS
# or: venv\Scripts\activate  # Windows

# 3. Install dependencies
pip install -r requirements.txt
```

Dependencies (requirements.txt)

```
# Core trading
py-clob-client>=0.34.0      # Polymarket CLOB API client
requests>=2.31.0             # HTTP client for Wethr.net + METAR APIs
python-dotenv>=1.0.0         # Environment variable management

# AI fair value estimation
anthropic>=0.39.0            # Claude API for probability estimation

# Scheduling and monitoring
schedule>=1.2.0              # Cron-like scheduling in Python

# Data storage and analysis
# (SQLite is built into Python - no extra dependency)
```

Environment Configuration (.env)

Create a `.env` file in the project root:

```
#
# Trading System Environment Configuration
#

#   Wethr.net Pro API
# Sign up at wethr.net/pro - this is your resolution-matching data source.
# Without this, you're trading blind to what WU will actually display.
WETHR_API_KEY=your_wethr_api_key_here

#   Polymarket
# Export your wallet's private key from MetaMask or your preferred wallet.
# Fund with USDC on Polygon network.
POLYMARKET_PRIVATE_KEY=your_polygon_wallet_private_key
POLYMARKET_FUNDER=your_wallet_address_0x...
POLYMARKET_SIGNATURE_TYPE=1

#   Polymarket API Endpoints
CLOB_ENDPOINT=https://clob.polymarket.com
GAMMA_ENDPOINT=https://gamma-api.polymarket.com

#   AI Model
# Haiku for monitoring (cheap), Sonnet for trade decisions (accurate)
ANTHROPIC_API_KEY=sk-ant-your-key-here
CLAUDE_MODEL=claude-3-haiku-20240307

#   Trading Parameters
MIN_EDGE_PERCENT=8.0             # Minimum edge to consider a trade
MAX_BET_SIZE=5.0                 # Max bet in USDC per market contract
```

```
MAX_TOTAL_RISK=100.0            # Max total capital at risk
SCAN_INTERVAL_MINUTES=10        # How often to scan for opportunities
MIN_LIQUIDITY=5000              # Minimum market liquidity (USDC)
MIN_VOLUME=2000                 # Minimum 24h volume

#   Notifications
# Uncomment and fill to enable Telegram push from Python scripts
# TELEGRAM_BOT_TOKEN=your_bot_token
# TELEGRAM_CHAT_ID=your_chat_id

#   Logging
LOG_LEVEL=INFO
```

Project Structure

```
polymarket-trading-bot/

   .env                              # API keys and config (DO NOT COMMIT)
   .gitignore                        # Excludes .env, venv/, data/
   requirements.txt                  # Python dependencies

   best_trades.py                    # Quick: top 2 opportunities now
   wethr_summary.py                  # Quick: all 6 cities WU + NWS
   start_trading.sh                  # Launch script for all components

   src/
       __init__.py
       wethr_client.py               # Wethr.net API client (REST + Push SSE)
       metar_cusp_model.py           # Cusp model + oscillation detection
       metar_aggressive_daemon.py    # Real-time METAR polling daemon
       metar_fetcher.py              # Direct METAR API client (aviationweather.gov)
       wu_rounding.py                # WU rounding safety calculator
       weather_gates.py              # Trade-blocking weather gates
       position_sizer.py             # Risk-based position sizing
       trade_executor.py             # Decision engine (pipeline orchestrator)
       best_trades_query.py          # Market scanner + ranking logic
       trend_analysis.py             # Temperature trend analyzer
       live_trader.py                # Live trading orchestrator
       polymarket_fetcher.py         # Polymarket market data client
       signal_validator.py           # Multi-source signal validation

   data/
       decisions.db                  # Trade decision log (SQLite)
       market_replay.db             # Historical price + METAR data

   config/
       stations.json                 # Station → city mapping
       brackets.json                 # Active bracket definitions
```

```
tests/
    test_wu_rounding.py           # WU rounding calculator tests
    test_position_sizer.py        # Position sizer tests (inc. Feb 15)
    test_weather_gates.py         # Gate logic tests
    test_cusp_model.py           # Cusp model tests

logs/                             # Runtime logs
    .gitkeep
```

Verify Installation

Run these in order. All must pass before trading:

```
# 1. Module imports
python3 -c "
from src.wethr_client import WethrClient
from src.metar_cusp_model import METARCuspModel
from src.weather_gates import WeatherGates
from src.wu_rounding import WURoundingCalculator
from src.position_sizer import PositionSizer
print('  All modules loaded successfully')
"

# 2. METAR access
python3 -c "
from src.metar_fetcher import METARFetcher
fetcher = METARFetcher()
metar = fetcher.get_latest('KATL')
if metar:
    print(f'  METAR working: KATL latest obs at {metar[\"obs_time\"]}')
else:
    print('  METAR fetch failed - check internet connection')
"

# 3. Wethr.net API
python3 wethr_summary.py

# 4. Feb 15 regression test - WU Rounding
python3 -c "
from src.wu_rounding import WURoundingCalculator
calc = WURoundingCalculator()
result = calc.feb15_validation()
print(result['VALIDATION'])
# Must print:  PASSED: Would have prevented the \$300 loss
"

# 5. Feb 15 regression test - Position Sizer
```

```
python3 -c "
from src.position_sizer import PositionSizer
sizer = PositionSizer(bankroll=300)
result = sizer.feb15_validation()
print(result['VALIDATION'])
# Must print:  PASSED: Would have capped at \$42 (not \$300)
"
```

If either Feb 15 validation fails → DO NOT TRADE. The safety layer is broken. Fix it before putting capital at risk.

Part 8: Wethr.net Pro API Integration

Why Wethr.net Is Essential

Wethr.net is a specialized weather data API built specifically for traders who need resolution-matching data. When you query Wethr.net with `logic=wu`, the returned `wethr_high` is processed using Weather Underground's own high-tracking logic. This means:

- **What Wethr.net shows = what WU will display = what Polymarket resolves on**

Without Wethr.net, you're guessing what WU will display based on raw METAR data. With Wethr.net, you know.

The Wethr.net Client

```
#!/usr/bin/env python3
"""
wethr_client.py - Wethr.net Pro API client.

Provides:
  - REST: WU-logic highs, NWS forecasts, all-cities summary
  - Push SSE: Real-time streaming of new highs, observations, suspect temps

The WU-logic high from this API is the closest real-time proxy for what
Polymarket's resolution source (Weather Underground) will display.
"""

import os
import json
import requests
import threading
from typing import Optional, Callable, Dict, List
from datetime import datetime, timezone
from dotenv import load_dotenv
```

```python
load_dotenv()


class WethrClient:
    """REST client for Wethr.net Pro API."""

    BASE_URL = 'https://api.wethr.net/v1'

    # Our 6 trading stations
    STATIONS = ['KATL', 'KDAL', 'KMIA', 'KSEA', 'KLGA', 'KORD']

    STATION_NAMES = {
        'KATL': 'Atlanta',
        'KDAL': 'Dallas',
        'KMIA': 'Miami',
        'KSEA': 'Seattle',
        'KLGA': 'NYC',
        'KORD': 'Chicago'
    }

    def __init__(self, api_key: str = None):
        self.api_key = api_key or os.getenv('WETHR_API_KEY')
        if not self.api_key:
            raise ValueError(
                "WETHR_API_KEY not set. Sign up at wethr.net/pro"
            )
        self.session = requests.Session()
        self.session.headers['Authorization'] = f'Bearer {self.api_key}'
        self.session.headers['Accept'] = 'application/json'

    def get_wethr_high(self, station: str, logic: str = 'wu') -> dict:
        """
        Get the WU-logic high/low for a station.

        Args:
            station: ICAO code (e.g., 'KATL')
            logic: 'wu' for Weather Underground matching, 'nws' for NWS

        Returns:
            {
                'station': 'KATL',
                'wethr_high': 65,        # What WU displays as daily high
                'wethr_low': 45,         # What WU displays as daily low
                'time_of_high_utc': '2026-03-08T18:53:00Z',
                'time_of_low_utc': '2026-03-08T09:15:00Z',
                'logic': 'wu'
            }
```

```python
        """
        resp = self.session.get(
            f'{self.BASE_URL}/wethr-high',
            params={'station': station.upper(), 'logic': logic}
        )
        resp.raise_for_status()
        return resp.json()

    def get_nws_forecast(self, station: str) -> dict:
        """
        Get the NWS forecast for a station.

        Returns:
            {
                'station': 'KATL',
                'high': 65,
                'low': 45,
                'version': 12,
                'hourly_temps': [45, 44, 43, ..., 50],  # 24 hourly values
                'updated_utc': '2026-03-08T16:00:00Z'
            }
        """
        resp = self.session.get(
            f'{self.BASE_URL}/nws-forecast',
            params={'station': station.upper()}
        )
        resp.raise_for_status()
        return resp.json()

    def get_all_highs(self, logic: str = 'wu') -> dict:
        """Get WU-logic highs for all 6 trading stations at once."""
        results = {}
        for station in self.STATIONS:
            try:
                results[station] = self.get_wethr_high(station, logic)
            except Exception as e:
                results[station] = {'error': str(e)}
        return results

    def summary(self, logic: str = 'wu') -> str:
        """
        Print a formatted summary of all cities.
        This is what wethr_summary.py calls.
        """
        lines = []
        now = datetime.now(timezone.utc).strftime('%H:%M UTC')
        lines.append(f"  Wethr.net Highs (logic={logic}) - {now}")
```

```python
        for station in self.STATIONS:
            try:
                data = self.get_wethr_high(station, logic)
                name = self.STATION_NAMES[station]
                high = data.get('wethr_high', '?')
                low = data.get('wethr_low', '?')
                time_h = data.get('time_of_high_utc', '?')[:19]
                lines.append(
                    f"  {name} ({station}): High={high}°F  Low={low}°F  "
                    f"@{time_h}"
                )
            except Exception as e:
                lines.append(f"  {station}: ERROR - {e}")

        lines.append("")
        lines.append("  NWS Forecasts (today projected high/low):")

        for station in self.STATIONS:
            try:
                fc = self.get_nws_forecast(station)
                name = self.STATION_NAMES[station]
                lines.append(
                    f"  {name} ({station}): High={fc['high']}°F  "
                    f"Low={fc['low']}°F  (v{fc.get('version', '?')})"
                )
            except Exception as e:
                lines.append(f"  {station}: ERROR - {e}")

        output = '\n'.join(lines)
        print(output)
        return output


class WethrPushClient:
    """
    SSE (Server-Sent Events) client for Wethr.net Push API.

    Streams real-time events:
    - new_high: WU-logic daily high updated
    - new_low: WU-logic daily low updated
    - observation: New METAR observation processed
    - suspect_temp: Temperature reading rejected by quality control
    - nws_update: NWS forecast revision detected

    Usage:
        push = WethrPushClient(stations=['KATL', 'KMIA'])
        push.on('new_high', lambda d: print(f"New high: {d}"))
        push.on('observation', lambda d: print(f"Obs: {d}"))
```

```
    push.start()  # Non-blocking background thread
"""

PUSH_URL = 'https://api.wethr.net/v1/push'

def __init__(
    self,
    stations: List[str] = None,
    api_key: str = None
):
    self.api_key = api_key or os.getenv('WETHR_API_KEY')
    self.stations = stations or [
        'KATL', 'KDAL', 'KMIA', 'KSEA', 'KLGA', 'KORD'
    ]
    self._handlers = {}
    self._running = False
    self._thread = None

def on(self, event_type: str, handler: Callable):
    """Register an event handler."""
    if event_type not in self._handlers:
        self._handlers[event_type] = []
    self._handlers[event_type].append(handler)

def start(self):
    """Start the SSE listener in a background thread."""
    self._running = True
    self._thread = threading.Thread(
        target=self._listen, daemon=True
    )
    self._thread.start()

def stop(self):
    """Stop the SSE listener."""
    self._running = False

def _listen(self):
    """Internal SSE listener loop with auto-reconnect."""
    while self._running:
        try:
            resp = requests.get(
                self.PUSH_URL,
                params={
                    'stations': ','.join(self.stations),
                    'events': ','.join(self._handlers.keys())
                },
                headers={
                    'Authorization': f'Bearer {self.api_key}',
```

```
                    'Accept': 'text/event-stream'
                },
                stream=True,
                timeout=300
            )
            resp.raise_for_status()

            event_type = None
            data_lines = []

            for line in resp.iter_lines(decode_unicode=True):
                if not self._running:
                    break

                if line.startswith('event:'):
                    event_type = line[6:].strip()
                elif line.startswith('data:'):
                    data_lines.append(line[5:].strip())
                elif line == '':
                    # End of event
                    if event_type and data_lines:
                        data = json.loads('\n'.join(data_lines))
                        self._dispatch(event_type, data)
                    event_type = None
                    data_lines = []

        except Exception as e:
            if self._running:
                import time
                print(f"Push reconnecting ({e})...")
                time.sleep(5)

    def _dispatch(self, event_type: str, data: dict):
        """Dispatch an event to registered handlers."""
        for handler in self._handlers.get(event_type, []):
            try:
                handler(data)
            except Exception as e:
                print(f"Handler error ({event_type}): {e}")
```

Using the Wethr.net Client

Quick summary (what `wethr_summary.py` does):

```
#!/usr/bin/env python3
"""wethr_summary.py - Quick overview of all 6 cities."""

import sys
```

```
sys.path.insert(0, '.')
from src.wethr_client import WethrClient

client = WethrClient()
client.summary(logic='wu')
```

Single city deep dive:

```
from src.wethr_client import WethrClient

client = WethrClient()

# WU-logic high (resolution-matching)
wu = client.get_wethr_high('KATL', logic='wu')
print(f"WU High: {wu['wethr_high']}°F at {wu['time_of_high_utc']}")
print(f"WU Low:  {wu['wethr_low']}°F")

# NWS forecast (prediction layer)
fc = client.get_nws_forecast('KATL')
print(f"NWS Forecast High: {fc['high']}°F (v{fc['version']})")
print(f"NWS Forecast Low:  {fc['low']}°F")

# Hourly forecast profile
print("Hourly temps:")
for hour, temp in enumerate(fc['hourly_temps']):
    if 6 <= hour <= 21:
        print(f"  {hour:02d}:00 → {temp}°F")

# Gap analysis: where's the edge?
gap = fc['high'] - wu['wethr_high']
print(f"\nForecast vs Current WU High: {gap:+d}°F")
if gap > 3:
    print("  → Temperature expected to climb significantly")
    print("  → Look for bracket opportunities above current WU high")
elif gap < -2:
    print("  → Forecast may have been too warm")
    print("  → Current WU high may be the peak")
```

Real-Time Push Monitoring

The push stream is how you get sub-minute alerts on temperature changes:

```
#!/usr/bin/env python3
"""
wethr_push_monitor.py - Real-time monitoring via Wethr.net Push SSE.

Run in a tmux session for persistent operation:
  tmux new -d -s wethr-push "cd ~/polymarket-trading-bot && source venv/bin/activate && python
"""
```

```python
import sys
sys.path.insert(0, '.')
from src.wethr_client import WethrPushClient
from datetime import datetime, timezone


def on_new_high(data):
    """Fires when WU-logic daily high updates for a station."""
    ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
    station = data['station_code']
    value = data['value_f']
    prev = data.get('prev_value_f', '?')
    logic = data.get('logic', '?')
    print(
        f"[{ts}]  NEW HIGH {station}: {value}°F "
        f"(was {prev}°F) [logic={logic}]"
    )


def on_new_low(data):
    ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
    print(
        f"[{ts}]  NEW LOW {data['station_code']}: "
        f"{data['value_f']}°F"
    )


def on_observation(data):
    """Fires on every new METAR observation processed."""
    ts = datetime.now(timezone.utc).strftime('%H:%M:%S')

    # Check for suspect temperature (quality control rejection)
    if data.get('suspect_temperature'):
        msg = data['suspect_temperature'].get('message', 'unknown')
        print(f"[{ts}]  {data['station_code']}: SUSPECT TEMP - {msg}")
        return

    station = data['station_code']
    temp_f = data.get('temperature_fahrenheit')
    wu_high = data.get('wethr_high', {}).get('wu', {}).get('value_f')
    obs_time = data.get('observation_time_utc', '')[:16]
    print(
        f"[{ts}]  {station}: {temp_f}°F | "
        f"WU High: {wu_high}°F | {obs_time}"
    )
```

```python
def on_nws_update(data):
    """Fires when NWS revises its forecast."""
    ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
    station = data.get('station_code', '?')
    old_high = data.get('prev_high')
    new_high = data.get('new_high')
    delta = new_high - old_high if old_high and new_high else '?'
    print(
        f"[{ts}]  NWS UPDATE {station}: "
        f"{old_high}°F → {new_high}°F ({delta:+d}°F)"
    )


# Set up push client
push = WethrPushClient()
push.on('new_high', on_new_high)
push.on('new_low', on_new_low)
push.on('observation', on_observation)
push.on('nws_update', on_nws_update)

print("  Wethr.net Push stream active (Ctrl+C to stop)")
print("   Monitoring: KATL, KDAL, KMIA, KSEA, KLGA, KORD")
print()

push.start()

try:
    import time
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    push.stop()
    print("\nStopped.")
```

Part 9: METAR Data Streaming & Monitoring

The METAR Aggressive Daemon

The daemon is the real-time backbone. It detects new METAR observations within seconds of posting to `aviationweather.gov` — 30–60 minutes before the data reaches consumer weather apps.

How it works:

1. During market hours (10 AM – 9 PM EST), the daemon is active
2. Each UTC hour, METAR observations typically post between :52 and :58
3. During this window, the daemon polls `aviationweather.gov` every **2 seconds**

4. When a new observation is detected (different obs time from last seen):
    - Parses the T-group for 0.1°C precision temperature
    - Records to `/tmp/metar_alerts.jsonl`
    - Sends Telegram notification (if enabled for that city)
    - Updates local state for downstream consumers

Outside the METAR window, the daemon sleeps to avoid unnecessary API calls, waking up ~60 seconds before the next expected observation.

METAR Fetcher Implementation

```python
#!/usr/bin/env python3
"""
metar_fetcher.py - Direct METAR fetch from aviationweather.gov

The ADDS (Aviation Digital Data Service) provides raw METAR data
via a simple HTTP API. No API key required. Rate limit: be reasonable
(1 request/2 seconds is fine).

We fetch from two sources for redundancy:
  Primary:   aviationweather.gov/api/data/metar
  Fallback:  aviationweather.gov/cgi-bin/data/dataserver.php
"""

import re
import requests
from typing import Optional, Dict
from datetime import datetime, timezone


class METARFetcher:
    """Fetch and parse METAR observations from aviationweather.gov."""

    PRIMARY_URL = 'https://aviationweather.gov/api/data/metar'

    # T-group regex: T followed by 8 digits
    TGROUP_RE = re.compile(r'T(\d{8})')

    # Wind regex: dddssKT or dddssGggKT
    WIND_RE = re.compile(
        r'(\d{3}|VRB)(\d{2,3})(?:G(\d{2,3}))?KT'
    )

    def __init__(self):
        self.session = requests.Session()
        self.session.headers['User-Agent'] = 'WeatherTrading/1.0'

    def get_latest(self, station: str) -> Optional[Dict]:
```

```python
    """
    Fetch the latest METAR observation for a station.

    Returns parsed dict or None on failure.
    """
    try:
        resp = self.session.get(
            self.PRIMARY_URL,
            params={
                'ids': station.upper(),
                'format': 'raw',
                'taf': 'false',
                'hours': 2
            },
            timeout=10
        )
        resp.raise_for_status()

        raw = resp.text.strip()
        if not raw:
            return None

        # Take the most recent observation (first line)
        metar_line = raw.split('\n')[0].strip()
        return self.parse(metar_line)

    except Exception as e:
        print(f"METAR fetch error for {station}: {e}")
        return None

def parse(self, metar_str: str) -> Dict:
    """
    Parse a raw METAR string into a structured dict.

    Extracts: station, time, wind, T-group (precise temp),
    main temp/dewpoint, altimeter.
    """
    parts = metar_str.split()
    result = {
        'raw': metar_str,
        'station': parts[0] if parts else None,
    }

    # Observation time (DDHHMMz)
    for part in parts:
        if part.endswith('Z') and len(part) == 7 and part[:6].isdigit():
            result['obs_time'] = part
            result['obs_day'] = int(part[:2])
```

```
                result['obs_hour'] = int(part[2:4])
                result['obs_min'] = int(part[4:6])
                break

        # Wind
        wind_match = self.WIND_RE.search(metar_str)
        if wind_match:
            direction_str = wind_match.group(1)
            result['wind'] = {
                'direction': int(direction_str) if direction_str != 'VRB' else None,
                'variable': direction_str == 'VRB',
                'speed': int(wind_match.group(2)),
                'gust': int(wind_match.group(3)) if wind_match.group(3) else None
            }

        # T-group (precise temperature - THE critical field)
        tgroup_match = self.TGROUP_RE.search(metar_str)
        if tgroup_match:
            tg_str = 'T' + tgroup_match.group(1)
            tg = parse_tgroup(tg_str)
            if tg:
                result['tgroup'] = tg
                result['temp_c_precise'] = tg['temp_c']
                result['temp_f_precise'] = tg['temp_f']
                result['wu_display'] = tg['wu_display']

        # Main temp/dewpoint (whole Celsius - lower precision)
        for part in parts:
            if '/' in part and not part.startswith('A') and not part.startswith('SLP'):
                try:
                    temp_dew = part.split('/')
                    if len(temp_dew) == 2:
                        temp_str = temp_dew[0].replace('M', '-')
                        dew_str = temp_dew[1].replace('M', '-')
                        result['temp_c_main'] = int(temp_str)
                        result['dewpoint_c'] = int(dew_str)
                except (ValueError, IndexError):
                    pass

        # Altimeter
        for part in parts:
            if part.startswith('A') and len(part) == 5 and part[1:].isdigit():
                result['altimeter_inhg'] = int(part[1:]) / 100.0

        return result


def parse_tgroup(tgroup_str: str) -> Optional[Dict]:
```

```python
    """Parse METAR T-group for precise temperature (0.1°C)."""
    if not tgroup_str or len(tgroup_str) != 9 or tgroup_str[0] != 'T':
        return None

    try:
        temp_sign = 1 if tgroup_str[1] == '0' else -1
        temp_c = temp_sign * int(tgroup_str[2:5]) / 10.0

        dew_sign = 1 if tgroup_str[5] == '0' else -1
        dewpoint_c = dew_sign * int(tgroup_str[6:9]) / 10.0

        temp_f = temp_c * 9.0 / 5.0 + 32.0

        return {
            'temp_c': temp_c,
            'temp_f': round(temp_f, 2),
            'dewpoint_c': dewpoint_c,
            'wu_display': round(temp_f),
            'raw': tgroup_str
        }
    except (ValueError, IndexError):
        return None
```

The Aggressive Daemon

```python
#!/usr/bin/env python3
"""
metar_aggressive_daemon.py - Sub-10-second METAR detection.

This daemon gives you the fastest possible access to new METAR observations.
Most weather apps update every 30-60 minutes. This detects new data in <10 seconds.

Architecture:
  - During the METAR window (:52-:58 UTC each hour), poll every 2 seconds
  - Outside the window, sleep until the next window
  - On new observation: parse, alert, log

Run modes:
  python3 src/metar_aggressive_daemon.py          # Production (continuous)
  python3 src/metar_aggressive_daemon.py --test    # Test (one cycle)

For persistent operation, run in tmux:
  tmux new -d -s metar "cd ~/polymarket-trading-bot && source venv/bin/activate && python3 src/
"""

import os
import sys
import json
```

```python
import time
import argparse
from datetime import datetime, timezone, timedelta
from typing import Dict, Optional

sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from src.metar_fetcher import METARFetcher, parse_tgroup


# Trading stations
STATIONS = {
    'KATL': 'Atlanta',
    'KDAL': 'Dallas',
    'KMIA': 'Miami',
    'KSEA': 'Seattle',
    'KLGA': 'NYC',
    'KORD': 'Chicago'
}

# Market hours: 10 AM - 9 PM EST (15:00 - 02:00+1 UTC)
MARKET_START_UTC = 15
MARKET_END_UTC = 2  # Next day

# METAR window within each hour
METAR_WINDOW_START = 52  # :52 past
METAR_WINDOW_END = 58    # :58 past

# Polling interval during METAR window
POLL_INTERVAL_SEC = 2

# Alert output file
ALERTS_FILE = '/tmp/metar_alerts.jsonl'

# Notification config file
NOTIF_CONFIG = '/tmp/metar_notifications.json'


class METARDaemon:
    def __init__(self, test_mode: bool = False):
        self.fetcher = METARFetcher()
        self.test_mode = test_mode
        self.last_obs = {}  # station → last obs_time seen
        self.notif_config = self._load_notif_config()

    def _load_notif_config(self) -> Dict:
        """Load per-city notification config."""
        try:
            with open(NOTIF_CONFIG) as f:
```

```
            return json.load(f)
    except FileNotFoundError:
        return {'enabled': False, 'cities': {}}

def _is_market_hours(self) -> bool:
    """Check if we're in market hours (10 AM - 9 PM EST)."""
    now = datetime.now(timezone.utc)
    hour = now.hour
    if MARKET_START_UTC <= hour <= 23:
        return True
    if 0 <= hour <= MARKET_END_UTC:
        return True
    return False

def _is_metar_window(self) -> bool:
    """Check if we're in the METAR observation window (:52-:58)."""
    now = datetime.now(timezone.utc)
    return METAR_WINDOW_START <= now.minute <= METAR_WINDOW_END

def _seconds_to_next_window(self) -> int:
    """Calculate seconds until next :52."""
    now = datetime.now(timezone.utc)
    if now.minute < METAR_WINDOW_START:
        target = now.replace(
            minute=METAR_WINDOW_START, second=0, microsecond=0
        )
    else:
        target = (now + timedelta(hours=1)).replace(
            minute=METAR_WINDOW_START, second=0, microsecond=0
        )
    return max(0, int((target - now).total_seconds()))

def poll_all_stations(self) -> Dict:
    """Poll all stations and detect new observations."""
    alerts = {}

    for station, city in STATIONS.items():
        try:
            metar = self.fetcher.get_latest(station)
            if not metar:
                continue

            obs_time = metar.get('obs_time', '')

            # Is this a new observation?
            if obs_time and obs_time != self.last_obs.get(station):
                self.last_obs[station] = obs_time
```

```
                # Build alert
                alert = {
                    'timestamp': datetime.now(timezone.utc).isoformat(),
                    'station': station,
                    'city': city,
                    'obs_time': obs_time,
                }

                if metar.get('tgroup'):
                    tg = metar['tgroup']
                    alert['temp_c'] = tg['temp_c']
                    alert['temp_f'] = tg['temp_f']
                    alert['wu_display'] = tg['wu_display']

                    ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
                    notif_on = self.notif_config.get(
                        'cities', {}
                    ).get(city, False)

                    status = '  ' if notif_on else ' '
                    notif_label = '' if notif_on else ' [notif OFF]'
                    print(
                        f"[{ts} UTC] {status} {city} "
                        f"{tg['temp_f']:.1f}°F ({tg['temp_c']}°C)"
                        f"{notif_label}"
                    )

                if metar.get('wind'):
                    alert['wind'] = metar['wind']

                # Write to alerts file
                with open(ALERTS_FILE, 'a') as f:
                    f.write(json.dumps(alert) + '\n')

                alerts[station] = alert

            except Exception as e:
                print(f"  Error polling {station}: {e}")

    return alerts

def run(self):
    """Main daemon loop."""
    print("=" * 72)
    print("  METAR AGGRESSIVE DAEMON STARTED")
    print(f"   Market hours: 10:00 - 21:00 EST")
    print(f"   METAR window: :{METAR_WINDOW_START}-:{METAR_WINDOW_END} UTC each hour")
    print(f"   Alerts file:  {ALERTS_FILE}")
```

```
        print(f"   Test mode:    {'YES' if self.test_mode else 'NO'}")
        ts = datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M:%S')
        print(f"   Started:      {ts} UTC")
        print("=" * 72)
        print()

        if self.test_mode:
            # Single poll cycle
            print("[TEST MODE] Running single poll cycle...")
            alerts = self.poll_all_stations()
            print(f"\n[TEST MODE] Got {len(alerts)} alerts. Exiting.")
            return

        while True:
            if not self._is_market_hours():
                print("  Outside market hours. Sleeping 10 min...")
                time.sleep(600)
                continue

            if self._is_metar_window():
                ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
                minute = datetime.now(timezone.utc).minute
                print(f"[{ts} UTC]   METAR window (:{minute:02d}) - polling every {POLL_INTERVA
                self.poll_all_stations()
                time.sleep(POLL_INTERVAL_SEC)
            else:
                secs = self._seconds_to_next_window()
                ts = datetime.now(timezone.utc).strftime('%H:%M:%S')
                print(f"[{ts} UTC]   Waiting {secs//60}m {secs%60}s for next METAR window...")
                time.sleep(min(secs, 60))  # Wake every minute to check market hours


if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--test', action='store_true', help='Run one cycle and exit')
    args = parser.parse_args()

    daemon = METARDaemon(test_mode=args.test)
    daemon.run()
```

Running the Daemon in tmux

The daemon must run continuously during market hours. Use tmux so it survives terminal disconnects and gateway restarts:

```
# Start the daemon in a named tmux session
tmux new-session -d -s metar-daemon \
  "cd ~/polymarket-trading-bot && \
```

```
   source venv/bin/activate && \
   python3 src/metar_aggressive_daemon.py; \
   echo 'EXITED:' \$?; sleep 999999"

# Check on it
tmux capture-pane -t metar-daemon -p | tail -20

# Kill when done for the day
tmux kill-session -t metar-daemon
```

Sample daemon output during a live METAR window:

```
======================================================================
  METAR AGGRESSIVE DAEMON STARTED
   Market hours: 10:00 - 21:00 EST
   METAR window: :52-:58 UTC each hour
   Alerts file:  /tmp/metar_alerts.jsonl
   Test mode:    NO
   Started:      2026-03-08 18:50:12 UTC
======================================================================

[18:50:14 UTC]  Waiting 1m 48s for next METAR window...
[18:51:14 UTC]  Waiting 0m 48s for next METAR window...
[18:52:03 UTC]  METAR window (:52) - polling every 2s
[18:52:03 UTC]  Atlanta 57.9°F (14.4°C) [notif OFF]
[18:52:03 UTC]  Dallas 68.0°F (20.0°C) [notif OFF]
[18:52:03 UTC]    Miami 79.0°F (26.1°C)
[18:52:05 UTC]  METAR window (:52) - polling every 2s
[18:52:07 UTC]  METAR window (:52) - polling every 2s
[18:53:01 UTC]  METAR window (:53) - polling every 2s
[18:53:01 UTC]    Chicago 42.1°F (5.6°C)  ← NEW observation detected
[18:53:03 UTC]  METAR window (:53) - polling every 2s
```

The daemon detected Chicago's :53 observation at `18:53:01` — 1 minute after it posted. Consumer weather apps typically show this data 30–60 minutes later.

Part 10: Daily Automation Workflows

The Complete Automation Stack

A trading operation that requires constant manual attention isn't an operation — it's a second job. The automation stack turns the system into something closer to a business: you set parameters, review opportunities, approve trades, and the machinery handles execution and monitoring.

Here's the full automation architecture:

```
Cron (system scheduler)
```

```
  7:45 AM EST  → Morning prep: fetch overnight WU highs, NWS updates
  8:00 AM EST  → Morning briefing push to Telegram
  Every 30 min → Heartbeat: METAR + gates + scanner + alerts
  9:00 PM EST  → Evening review: P&L, resolutions, next-day preview
  11:59 PM EST → Midnight cleanup: rotate logs, archive today's data

tmux sessions (persistent)
  metar-daemon → Real-time METAR monitoring
  wethr-push   → Wethr.net SSE push stream
  scanner      → Opportunity scanner (every 10 minutes)

OpenClaw heartbeats (AI-driven)
  Every 30 min → AI analysis of METAR data, trading recommendations
```

Setting Up Cron Jobs

Cron handles time-based scheduling. Each line in your crontab represents a scheduled task.

```
# Open your crontab
crontab -e
```

Add the following lines (adjust paths as needed):

```
#
# Polymarket Weather Trading - Cron Schedule
#
# Path setup for virtual environment
SHELL=/bin/bash
PATH=/usr/local/bin:/usr/bin:/bin:/home/USER/.local/bin
TRADING_DIR=/home/USER/polymarket-trading-bot
VENV=source /home/USER/polymarket-trading-bot/venv/bin/activate

#   Morning Prep (7:45 AM EST = 12:45 UTC)
45 12 * * * cd $TRADING_DIR && source venv/bin/activate && python3 wethr_summary.py >> logs/mor

#   Morning Briefing (8:00 AM EST = 13:00 UTC)
0 13 * * * cd $TRADING_DIR && source venv/bin/activate && python3 scripts/morning_brief.py >> l

#   Scanner: Every 10 minutes, 10 AM - 9 PM EST (15:00 - 02:00 UTC)
*/10 15-23 * * * cd $TRADING_DIR && source venv/bin/activate && python3 best_trades.py >> logs/
*/10 0-2 * * * cd $TRADING_DIR && source venv/bin/activate && python3 best_trades.py >> logs/sc

#   Heartbeat: Every 30 minutes during market hours
*/30 15-23 * * * cd $TRADING_DIR && source venv/bin/activate && python3 scripts/heartbeat.py >>
*/30 0-2 * * * cd $TRADING_DIR && source venv/bin/activate && python3 scripts/heartbeat.py >> l

#   Evening Review (9:00 PM EST = 02:00 UTC next day)
0 2 * * * cd $TRADING_DIR && source venv/bin/activate && python3 scripts/evening_review.py >> l
```

```
#   Midnight Cleanup (11:59 PM EST = 04:59 UTC)
59 4 * * * cd $TRADING_DIR && bash scripts/daily_cleanup.sh >> logs/cleanup.log 2>&1

#   Daemon Health Check (every 5 minutes)
*/5 * * * * bash /home/USER/polymarket-trading-bot/scripts/check_daemons.sh >> logs/daemon-heal
```

The Daemon Health Check Script

Daemons crash. This script detects and restarts them automatically:

```bash
#!/bin/bash
# check_daemons.sh - Verify daemons are running, restart if not

TRADING_DIR="$HOME/polymarket-trading-bot"
VENV="$TRADING_DIR/venv/bin/activate"
LOG="$TRADING_DIR/logs/daemon-health.log"
TMUX_SOCKET="$HOME/.tmux/sock"

ts() { date '+%Y-%m-%d %H:%M:%S'; }

check_and_restart() {
    local session_name=$1
    local command=$2

    if tmux -S "$TMUX_SOCKET" has-session -t "$session_name" 2>/dev/null; then
        echo "$(ts)   $session_name: running"
    else
        echo "$(ts)    $session_name: NOT running - restarting"
        tmux -S "$TMUX_SOCKET" new-session -d -s "$session_name" "$command"
        echo "$(ts)   $session_name: restarted"
    fi
}

# Check METAR daemon
check_and_restart "metar-daemon" \
    "cd $TRADING_DIR && source $VENV && python3 src/metar_aggressive_daemon.py; echo 'EXITED:'

# Check Wethr.net push stream
check_and_restart "wethr-push" \
    "cd $TRADING_DIR && source $VENV && python3 wethr_push_monitor.py; echo 'EXITED:' \$?; slee
```

The Morning Briefing Script

```python
#!/usr/bin/env python3
"""
scripts/morning_brief.py - 8:00 AM briefing generator.
```

```python
Gathers all relevant data and pushes a structured briefing to Telegram.
Designed to give the trader everything needed to plan the day's trades.
"""

import os
import sys
from datetime import datetime, timezone

sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from src.wethr_client import WethrClient
from src.weather_gates import WeatherGates

STATIONS = {
    'KATL': 'Atlanta', 'KDAL': 'Dallas', 'KMIA': 'Miami',
    'KSEA': 'Seattle', 'KLGA': 'NYC', 'KORD': 'Chicago'
}


def generate_morning_brief() -> str:
    client = WethrClient()
    now_est = datetime.now().strftime('%A, %B %-d, %Y')

    lines = [
        f"  **MORNING BRIEFING - {now_est}**",
        "",
        "  **WU Highs (overnight)**"
    ]

    all_highs = client.get_all_highs(logic='wu')
    for station, data in all_highs.items():
        city = STATIONS[station]
        if 'error' not in data:
            high = data.get('wethr_high', '?')
            low = data.get('wethr_low', '?')
            lines.append(f"  {city}: {high}°F / {low}°F")
        else:
            lines.append(f"  {city}:  Data unavailable")

    lines += ["", "  **NWS Forecast Highs (today)**"]

    for station in STATIONS:
        city = STATIONS[station]
        try:
            fc = client.get_nws_forecast(station)
            high = fc.get('high', '?')
            low = fc.get('low', '?')
            version = fc.get('version', '?')
            lines.append(f"  {city}: {high}°F forecast high (v{version})")
```

```
        except Exception:
            lines.append(f"  {city}:  Forecast unavailable")

    lines += [
        "",
        "  **Action Items**",
        "  • Run `python3 best_trades.py` for today's opportunities",
        "  • Check daemon health: `tmux ls`",
        "  • Review yesterday's P&L in decisions.db",
        "",
        "  METAR daemon and Wethr.net push stream should be running.",
        "   Check with: tmux -S ~/.tmux/sock ls",
    ]

    return '\n'.join(lines)


if __name__ == '__main__':
    brief = generate_morning_brief()
    print(brief)

    # Push to Telegram if configured
    telegram_token = os.getenv('TELEGRAM_BOT_TOKEN')
    telegram_chat = os.getenv('TELEGRAM_CHAT_ID')

    if telegram_token and telegram_chat:
        import requests
        resp = requests.post(
            f'https://api.telegram.org/bot{telegram_token}/sendMessage',
            json={
                'chat_id': telegram_chat,
                'text': brief,
                'parse_mode': 'Markdown'
            }
        )
        if resp.ok:
            print("  Briefing sent to Telegram")
        else:
            print(f"  Telegram send failed: {resp.text}")
```

The Heartbeat Script

```
#!/usr/bin/env python3
"""
scripts/heartbeat.py - 30-minute automated check.

Checks all open positions for safety margin breaches,
scans for new opportunities, and sends alerts via Telegram.
```

```python
"""

import os
import sys
import json
import sqlite3
from datetime import datetime, timezone

sys.path.insert(0, os.path.join(os.path.dirname(__file__), '..'))
from src.metar_fetcher import METARFetcher
from src.wu_rounding import WURoundingCalculator
from src.weather_gates import WeatherGates


def run_heartbeat():
    ts = datetime.now(timezone.utc).strftime('%H:%M UTC')
    print(f"[{ts}]  Heartbeat starting...")

    fetcher = METARFetcher()
    calc = WURoundingCalculator()
    alerts = []

    # Load open positions from DB
    conn = sqlite3.connect('data/decisions.db')
    cursor = conn.execute("""
        SELECT station, city, bracket_lower, bracket_upper, direction, size_usdc
        FROM decisions
        WHERE outcome = 'PENDING'
        ORDER BY entry_time DESC
    """)
    open_positions = cursor.fetchall()
    conn.close()

    for station, city, bl, bu, direction, size in open_positions:
        metar = fetcher.get_latest(station)
        if not metar or not metar.get('tgroup'):
            alerts.append(f"  {city}: No METAR data")
            continue

        actual_f = metar['tgroup']['temp_f']
        margin_result = calc.calculate(actual_f, bl, bu, direction)
        margin = margin_result['effective_margin']

        if margin < 0.2:
            alerts.append(
                f"  EXIT NOW - {city} {bl}-{bu}°F {direction}\n"
                f"    Margin: {margin:.2f}°F (EXTREME RISK)\n"
                f"    Current: {actual_f:.1f}°F | WU: {round(actual_f)}°F"
```

```
            )
        elif margin < 0.3:
            alerts.append(
                f"  CAUTION - {city} {bl}-{bu}°F {direction}\n"
                f"    Margin: {margin:.2f}°F (HIGH RISK)\n"
                f"    Current: {actual_f:.1f}°F"
            )

    # Print and optionally push alerts
    for alert in alerts:
        print(alert)

    if not alerts:
        print(f"[{ts}]   All positions within safety margins")

    print(f"[{ts}]   Heartbeat complete. {len(open_positions)} open positions checked.")


if __name__ == '__main__':
    run_heartbeat()
```

The Daily Cleanup Script

```
#!/bin/bash
# scripts/daily_cleanup.sh - Midnight log rotation and archiving

TRADING_DIR="$HOME/polymarket-trading-bot"
DATE=$(date +%Y-%m-%d)
ARCHIVE="$TRADING_DIR/logs/archive/$DATE"

mkdir -p "$ARCHIVE"

# Archive today's logs
for log in scanner.log heartbeat.log morning-brief.log; do
    if [ -f "$TRADING_DIR/logs/$log" ]; then
        cp "$TRADING_DIR/logs/$log" "$ARCHIVE/$log"
        echo "" > "$TRADING_DIR/logs/$log"  # Clear (don't delete)
        echo "  Archived $log"
    fi
done

# Archive today's METAR alerts
if [ -f "/tmp/metar_alerts.jsonl" ]; then
    cp /tmp/metar_alerts.jsonl "$ARCHIVE/metar_alerts.jsonl"
    echo "" > /tmp/metar_alerts.jsonl
    echo "  Archived metar_alerts.jsonl"
fi
```

```
# Prune archives older than 90 days
find "$TRADING_DIR/logs/archive" -type d -mtime +90 -exec rm -rf {} + 2>/dev/null

echo "  Daily cleanup complete - $(date)"
```

Start Everything: The Master Launch Script

```
#!/bin/bash
# start_trading.sh - Launch all trading system components

set -e
TRADING_DIR="$(cd "$(dirname "$0")" && pwd)"
VENV="$TRADING_DIR/venv/bin/activate"
TMUX_SOCKET="$HOME/.tmux/sock"

echo "  Starting trading system..."

# 1. Ensure virtual environment exists
if [ ! -f "$VENV" ]; then
    echo "  Virtual environment not found. Run: python3 -m venv venv && pip install -r requirem
    exit 1
fi

# 2. Verify installation
echo "Checking installation..."
source "$VENV"
python3 -c "from src.wethr_client import WethrClient; print('  Core modules OK')"

# 3. Start METAR daemon
echo "Starting METAR daemon..."
tmux -S "$TMUX_SOCKET" new-session -d -s metar-daemon \
    "cd $TRADING_DIR && source $VENV && \
     python3 src/metar_aggressive_daemon.py; \
     echo 'EXITED:' \$?; sleep 999999"

# 4. Start Wethr.net push stream
echo "Starting Wethr.net push stream..."
tmux -S "$TMUX_SOCKET" new-session -d -s wethr-push \
    "cd $TRADING_DIR && source $VENV && \
     python3 wethr_push_monitor.py; \
     echo 'EXITED:' \$?; sleep 999999"

# 5. Print status
echo ""
echo "  Trading system started!"
echo ""
echo "Active tmux sessions:"
tmux -S "$TMUX_SOCKET" list-sessions
```

```
echo ""
echo "Quick commands:"
echo "  Monitor METAR:  tmux -S ~/.tmux/sock attach -t metar-daemon"
echo "  Monitor Push:   tmux -S ~/.tmux/sock attach -t wethr-push"
echo "  Get top trades: python3 best_trades.py"
echo "  City summary:   python3 wethr_summary.py"
```

Part 11: The Decision Database — Schema & Pattern Analysis

Why Log Everything

The decision database is the trading operation's institutional memory. Every trade decision, every signal, every gate check gets logged. This creates the raw material for pattern extraction — the only way to systematically improve over time.

Without a database, you're relying on memory and intuition. With one, you can run queries like:

- "What's my win rate on Chicago trades when entry is before noon?"
- "Which gate fires most often? How accurate is it?"
- "What's my average edge on trades I actually take versus trades I skip?"

The database turns experience into evidence.

Schema

```sql
-- decisions.db

--   Core trade decisions
CREATE TABLE IF NOT EXISTS decisions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,

    -- Market context
    station        TEXT NOT NULL,        -- ICAO station code (e.g., KATL)
    city           TEXT NOT NULL,        -- Human name (Atlanta)
    market_id      TEXT,                 -- Polymarket market ID
    bracket_lower  REAL NOT NULL,        -- Bracket lower bound (°F)
    bracket_upper  REAL NOT NULL,        -- Bracket upper bound (°F)
    direction      TEXT NOT NULL,        -- YES or NO

    -- Price and edge
    market_price   REAL,                 -- Market price at decision time (0-1)
    fair_value     REAL,                 -- Estimated fair value (0-1)
    edge_percent   REAL,                 -- (fair_value - market_price) / market_price * 100
```

```sql
    -- Weather data at decision time
    metar_temp_f   REAL,                 -- METAR T-group temp (°F)
    metar_temp_c   REAL,                 -- METAR T-group temp (°C)
    metar_obs_time TEXT,                 -- METAR observation timestamp
    wu_display     INTEGER,              -- What WU shows right now
    wu_high_current REAL,               -- Current WU daily high
    nws_forecast_f REAL,                -- NWS forecast high for today

    -- Risk metrics
    safety_margin  REAL,                 -- WU rounding safety margin (°F)
    risk_level     TEXT,                 -- LOW / MEDIUM / HIGH / EXTREME

    -- Gate results
    sea_breeze_blocked INTEGER DEFAULT 0,
    marine_layer_caution INTEGER DEFAULT 0,
    forecast_divergence_blocked INTEGER DEFAULT 0,
    polar_vortex_blocked INTEGER DEFAULT 0,
    gates_json     TEXT,                 -- Full gate results as JSON

    -- Position sizing
    position_usdc  REAL,                 -- Recommended position size
    actual_usdc    REAL,                 -- Actual position taken (0 if skipped)

    -- Decision metadata
    signal_type    TEXT,                 -- Signal identifier (safe_bullish, etc.)
    confidence     TEXT,                 -- HIGH / MEDIUM / LOW
    data_sources   INTEGER DEFAULT 0,   -- Number of confirming sources
    entry_time     TEXT,                 -- ISO timestamp

    -- Outcome
    outcome        TEXT DEFAULT 'PENDING',  -- WIN / LOSS / SKIPPED / PENDING
    resolution_f   REAL,                    -- Actual resolved temperature (°F)
    pnl_usdc       REAL,                    -- P&L in USDC
    exit_time      TEXT                     -- ISO timestamp
);

--   Gate tracking
CREATE TABLE IF NOT EXISTS gate_triggers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    decision_id INTEGER REFERENCES decisions(id),
    gate_name    TEXT NOT NULL,          -- sea_breeze / marine_layer / etc.
    blocked      INTEGER NOT NULL,       -- 1 = hard block, 0 = caution
    reason       TEXT,
    triggered_at TEXT DEFAULT (datetime('now'))
);

--   METAR observations log
CREATE TABLE IF NOT EXISTS metar_log (
```

```sql
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    station      TEXT NOT NULL,
    obs_time     TEXT NOT NULL,
    temp_c       REAL,
    temp_f       REAL,
    wu_display   INTEGER,
    wind_dir     INTEGER,
    wind_speed   INTEGER,
    wind_gust    INTEGER,
    tgroup_raw   TEXT,
    logged_at    TEXT DEFAULT (datetime('now'))
);

--   Useful indexes
CREATE INDEX IF NOT EXISTS idx_decisions_city ON decisions(city);
CREATE INDEX IF NOT EXISTS idx_decisions_outcome ON decisions(outcome);
CREATE INDEX IF NOT EXISTS idx_decisions_entry_time ON decisions(entry_time);
CREATE INDEX IF NOT EXISTS idx_metar_station ON metar_log(station, obs_time);
```

Pattern Extraction Queries

These queries are your retrospective edge discovery tools. Run them weekly.

Win rate by city:

```sql
SELECT
    city,
    COUNT(*) as total_trades,
    SUM(CASE WHEN outcome = 'WIN' THEN 1 ELSE 0 END) as wins,
    ROUND(100.0 * SUM(CASE WHEN outcome = 'WIN' THEN 1 ELSE 0 END) / COUNT(*), 1) as win_rate_p
    ROUND(SUM(pnl_usdc), 2) as total_pnl,
    ROUND(AVG(edge_percent), 1) as avg_edge_pct
FROM decisions
WHERE outcome IN ('WIN', 'LOSS')
GROUP BY city
ORDER BY total_pnl DESC;
```

Win rate by entry time of day:

```sql
SELECT
    CASE
        WHEN CAST(strftime('%H', entry_time) AS INTEGER) < 12 THEN 'Morning (before noon)'
        WHEN CAST(strftime('%H', entry_time) AS INTEGER) < 15 THEN 'Afternoon (noon-3PM)'
        ELSE 'Late (after 3PM)'
    END as time_of_day,
    COUNT(*) as trades,
    ROUND(100.0 * SUM(CASE WHEN outcome = 'WIN' THEN 1 ELSE 0 END) / COUNT(*), 1) as win_rate_p
    ROUND(SUM(pnl_usdc), 2) as pnl
FROM decisions
```

```sql
WHERE outcome IN ('WIN', 'LOSS')
GROUP BY time_of_day;
```

Safety margin vs outcome correlation:

```sql
SELECT
    CASE
        WHEN safety_margin >= 1.0 THEN ' 1.0°F (LOW risk)'
        WHEN safety_margin >= 0.5 THEN '0.5-1.0°F (MEDIUM risk)'
        WHEN safety_margin >= 0.3 THEN '0.3-0.5°F (HIGH risk)'
        ELSE '<0.3°F (EXTREME risk)'
    END as margin_tier,
    COUNT(*) as trades,
    ROUND(100.0 * SUM(CASE WHEN outcome = 'WIN' THEN 1 ELSE 0 END) / COUNT(*), 1) as win_rate_
    ROUND(AVG(pnl_usdc), 2) as avg_pnl_per_trade
FROM decisions
WHERE outcome IN ('WIN', 'LOSS')
GROUP BY margin_tier
ORDER BY MIN(safety_margin) DESC;
```

Gate accuracy — how often blocked trades would have lost:

```sql
-- Compare trades blocked by gates vs trades that passed
-- (requires manually tracking "would have outcome" for blocked trades)
SELECT
    gate_name,
    COUNT(*) as times_triggered,
    SUM(blocked) as hard_blocks,
    COUNT(*) - SUM(blocked) as cautions
FROM gate_triggers
GROUP BY gate_name;
```

Signal type performance:

```sql
SELECT
    signal_type,
    COUNT(*) as trades,
    ROUND(100.0 * SUM(CASE WHEN outcome = 'WIN' THEN 1 ELSE 0 END) / COUNT(*), 1) as win_rate,
    ROUND(SUM(pnl_usdc), 2) as total_pnl
FROM decisions
WHERE outcome IN ('WIN', 'LOSS')
GROUP BY signal_type
ORDER BY total_pnl DESC;
```

Logging a Decision

Every decision goes into the database before execution. This creates an immutable record of the thesis at the moment of entry — not the rationalized version you'd construct after knowing the outcome.

```python
def log_decision(conn: sqlite3.Connection, decision: dict) -> int:
    """
    Log a trade decision to the database before execution.

    Returns the decision ID (for later outcome updates).

    The 'before execution' part matters: you're recording your
    thesis as it existed when you made the call, not the story
    you tell yourself afterward.
    """
    cursor = conn.execute("""
        INSERT INTO decisions (
            station, city, market_id, bracket_lower, bracket_upper,
            direction, market_price, fair_value, edge_percent,
            metar_temp_f, metar_temp_c, wu_display, wu_high_current,
            nws_forecast_f, safety_margin, risk_level,
            sea_breeze_blocked, marine_layer_caution,
            forecast_divergence_blocked, polar_vortex_blocked, gates_json,
            position_usdc, actual_usdc, signal_type, confidence,
            data_sources, entry_time, outcome
        ) VALUES (
            :station, :city, :market_id, :bracket_lower, :bracket_upper,
            :direction, :market_price, :fair_value, :edge_percent,
            :metar_temp_f, :metar_temp_c, :wu_display, :wu_high_current,
            :nws_forecast_f, :safety_margin, :risk_level,
            :sea_breeze_blocked, :marine_layer_caution,
            :forecast_divergence_blocked, :polar_vortex_blocked, :gates_json,
            :position_usdc, :actual_usdc, :signal_type, :confidence,
            :data_sources, :entry_time, 'PENDING'
        )
    """, decision)
    conn.commit()
    return cursor.lastrowid


def update_outcome(conn: sqlite3.Connection, decision_id: int,
                   outcome: str, resolution_f: float, pnl_usdc: float):
    """Update a pending decision with its resolution."""
    conn.execute("""
        UPDATE decisions
        SET outcome = ?, resolution_f = ?, pnl_usdc = ?, exit_time = datetime('now')
        WHERE id = ?
    """, (outcome, resolution_f, pnl_usdc, decision_id))
    conn.commit()
```

Part 12: The Complete Post-Mortem — Real Trades, Real Losses, Real Fixes

February 2026: The Learning Lab

The first 15 trades weren't a trading operation. They were expensive tuition. Here's the complete forensic analysis of every failure, what it cost, and what component we built to prevent it from happening again.

February 2026 Summary:

Metric	Value
Total trades	15
Wins	4 (26.7%)
Losses	11 (73.3%)
Total P&L	-$187
Largest single loss	-$82 (Feb 18, Miami pyramiding)
Largest single win	+$94 (Feb 16, Chicago)
Average edge (pre-trade estimate)	18.3%
Average actual outcome	-12.5%

The Failure Taxonomy

Category 1: WU Rounding Blindness (3 trades, -$112)

We calculated the distance from the current temperature to the bracket edge as a raw Fahrenheit number. We did not account for WU's rounding logic.

Example: Temperature at 57.02°F, bracket upper at 58°F. We calculated: "0.98°F of safety — plenty of room." WU's rounding threshold for 58 is 57.5°F. Actual safety: 0.48°F. The temperature reached 57.6°F. WU displayed 58°F. Trade lost.

Fix built: `wu_rounding.py` — calculates effective margin to WU rounding boundaries, not raw bracket edges. Self-validates against the Feb 15 scenario on every import.

Category 2: Panic Interpretation (3 trades, -$68)

One adverse METAR reading triggered an exit from a position that subsequently resolved as a win. We exited at a loss because we interpreted a single 5-minute reading as a trend rather than noise.

Example: Holding the 64–65°F bracket. METAR comes in at 64.9°F (WU: 65°F). Next reading: 64.2°F (WU: 64°F). We panic-exited. Three readings later: 65.4°F (WU: 65°F). Resolved as a win. We held for nothing.

Fix built: The oscillation detector in `metar_cusp_model.py`. Single adverse readings no longer trigger exits — the trend must be confirmed by 3 consecutive adverse readings, or by margin falling below 0.2°F.

Category 3: Pyramiding into Losses (2 trades, -$94)

When a position moved against us, we added to it rather than exiting. Classic mistake. A $100 position became $250 position at worse prices. Both resolved as losses.

Fix built: RULES.md hard rule: "Never add to a losing position." The decision database logs every add-on separately, creating accountability.

Category 4: Wrong Station (1 trade, -$32)

Dallas trade placed using KDFW temperature data. Market resolves on KDAL (Love Field). The two stations differed by 4°F that day. We had the right thesis, wrong data source.

Fix built: `config/stations.json` — the canonical station map with explicit warnings about Dallas. Every scan and query now validates station code before processing.

Category 5: Sea Breeze Ignored (1 trade, -$42)

Miami trade expecting 82°F. South wind at 185° with 18-knot gusts was already established. We noted the wind pattern but didn't block the trade because the forecast still called for 83°F. The sea breeze held. Temperature peaked at 79°F. Market resolved NO for the 80–81°F bracket.

Fix built: `weather_gates.py` Gate 1 (Sea Breeze Cap) — hard block, no override possible when the signature matches.

Category 6: Forecast Divergence (1 trade, -$26)

We entered a Seattle trade at 2 PM based on an 8 AM forecast calling for 68°F. By 2 PM, the actual temperature was stuck at 61°F. The forecast was clearly wrong — 7°F divergence over 6 hours — but we entered anyway, reasoning that "it might still warm up." It didn't.

Fix built: `weather_gates.py` Gate 3 (Forecast Divergence) — hard block when scanner prediction and actual observations diverge >1.5°F for >30 minutes.

What the Winners Had in Common

The 4 winning trades were not wins because we predicted the weather better. They were wins because of discipline:

1. **All entered before noon.** Morning entries are better because there's more remaining time for the thesis to play out and more temperature data to confirm the trend.
2. **All held to resolution.** Zero panic exits. The thesis was confirmed by multiple data sources, so we held.
3. **All had METAR confirmation.** Every winning trade had at least one T-group reading that confirmed the temperature was sitting comfortably inside the target bracket with margin > 0.8°F.
4. **All had 2+ data sources.** NWS forecast + Wethr.net WU high + METAR T-group. The confluence reduced uncertainty.

The Chicago Case Study: Why This Market Worked

Chicago went 2/2 in February. Both trades were held to resolution. P&L: +$112.

Why Chicago was more predictable:

KORD sits in a continental interior climate with minimal marine or sea breeze influence. The diurnal temperature cycle is driven almost entirely by solar radiation and air mass characteristics — both of which the NWS forecast models handle well. Without the sea breeze capping effects (Miami), marine layer complications (Seattle), or urban heat island complexities (NYC), KORD temperatures track NWS forecasts with less than 1°F average error in winter.

The Chicago edge in plain language: Better forecast accuracy means better edge estimation. Better edge estimation means better position sizing. Better position sizing means larger positions on genuine opportunities rather than coin-flip bets. That's where the +$112 came from.

Part 13: Advanced Monitoring & Alerts

Alert Architecture: Three Channels, One Priority System

Not all alerts are equal. The monitoring system routes alerts based on urgency:

```
Priority 1: URGENT (require immediate action within minutes)
   Safety margin below 0.2°F on open position → EXIT NOW
   Weather gate triggered on open position → EXIT NOW
   Suspected fraud/manipulation in market → PAUSE TRADING

Priority 2: IMPORTANT (review within 30 minutes)
   Safety margin 0.2-0.3°F → CAUTION
   NWS forecast revision > 3°F → RECHECK
   New high-edge opportunity (>20%) → CONSIDER ENTRY

Priority 3: INFORMATIONAL (check at next heartbeat)
   New METAR observation (all stations)
   Daily WU high updates
   Routine heartbeat status
```

Routing: - Priority 1 → Telegram push notification (immediate wake-up) - Priority 2 → Telegram message (standard notification) - Priority 3 → Log file only (no notification)

Telegram Alert Integration

```python
#!/usr/bin/env python3
"""
src/alert_system.py - Structured Telegram alert delivery.
```

```python
Provides priority-based routing, message formatting, and
rate limiting to prevent notification fatigue.
"""

import os
import time
import requests
from datetime import datetime, timezone
from typing import Optional


class AlertSystem:
    """
    Priority-based Telegram alert system.

    Rate limits:
    - URGENT: No rate limit (always send)
    - IMPORTANT: Max 1 per 5 minutes per city
    - INFO: Max 1 per 30 minutes total
    """

    PRIORITY_URGENT = 1
    PRIORITY_IMPORTANT = 2
    PRIORITY_INFO = 3

    def __init__(self):
        self.token = os.getenv('TELEGRAM_BOT_TOKEN')
        self.chat_id = os.getenv('TELEGRAM_CHAT_ID')
        self._last_sent = {}  # (priority, key) → timestamp

    def _rate_check(self, priority: int, key: str) -> bool:
        """Return True if we should send (passes rate limit check)."""
        if priority == self.PRIORITY_URGENT:
            return True

        limits = {
            self.PRIORITY_IMPORTANT: 300,   # 5 minutes
            self.PRIORITY_INFO: 1800        # 30 minutes
        }

        limit_secs = limits.get(priority, 1800)
        last = self._last_sent.get((priority, key), 0)
        return (time.time() - last) >= limit_secs

    def send(self, message: str, priority: int = PRIORITY_INFO,
             key: str = 'default') -> bool:
        """
        Send an alert.
```

```
        Args:
            message: Alert text (Markdown supported)
            priority: PRIORITY_URGENT, IMPORTANT, or INFO
            key: Rate-limit key (e.g., city name)

        Returns:
            True if sent, False if rate-limited or no config
        """
        if not self.token or not self.chat_id:
            print(f"[ALERT] {message}")
            return False

        if not self._rate_check(priority, key):
            return False

        # Priority prefix
        prefix = {
            self.PRIORITY_URGENT: "  *URGENT*",
            self.PRIORITY_IMPORTANT: "  *IMPORTANT*",
            self.PRIORITY_INFO: "  *INFO*"
        }.get(priority, "  *INFO*")

        full_message = f"{prefix}\n\n{message}"

        try:
            resp = requests.post(
                f'https://api.telegram.org/bot{self.token}/sendMessage',
                json={
                    'chat_id': self.chat_id,
                    'text': full_message,
                    'parse_mode': 'Markdown'
                },
                timeout=10
            )
            if resp.ok:
                self._last_sent[(priority, key)] = time.time()
                return True
        except Exception as e:
            print(f"Alert send failed: {e}")

        return False

    def exit_alert(self, city: str, bracket: str, margin: float, temp_f: float):
        """Send urgent exit alert."""
        self.send(
            f"EXIT IMMEDIATELY\n\n"
            f"City: {city}\n"
```

```
            f"Bracket: {bracket}\n"
            f"Margin: {margin:.2f}°F (EXTREME RISK)\n"
            f"Current temp: {temp_f:.1f}°F\n\n"
            f"Exit now at any price to preserve capital.",
            priority=self.PRIORITY_URGENT,
            key=city
        )

    def opportunity_alert(self, city: str, bracket: str, edge_pct: float,
                          price: float, size_usdc: float, margin: float):
        """Send opportunity alert."""
        self.send(
            f"New Opportunity\n\n"
            f"City: {city} - {bracket}\n"
            f"Edge: {edge_pct:.1f}%\n"
            f"Market price: {price:.2f}\n"
            f"Recommended size: ${size_usdc:.0f}\n"
            f"Safety margin: {margin:.2f}°F",
            priority=self.PRIORITY_IMPORTANT,
            key=city
        )
```

The Scanner Dashboard (Terminal UI)

For visual monitoring during active trading hours, the scanner dashboard provides a live overview of all cities:

```
          WEATHER TRADING MONITOR - 14:23 EST

City            WU High  NWS Fc   METAR    Margin   Gates

Atlanta          64°F     67°F    63.8°F    -        ALL
Dallas           72°F     74°F    71.2°F    1.3°F    ALL
Miami            78°F     83°F    77.4°F    -        SEA
Seattle          52°F     55°F    51.8°F    -        MARINE
NYC              58°F     61°F    57.9°F    -        ALL
Chicago          45°F     48°F    44.6°F    -        ALL

OPEN POSITIONS: 1
Dallas 70-71°F YES | $45 at risk | Margin: 1.3°F |   SAFE

OPPORTUNITIES (edge > 15%):
→ Atlanta 66-67°F YES: 22% edge | $75 rec | Wait for 66°F
```

Part 14: Scaling & Optimization

The Scaling Hierarchy

The system as built handles the current operation well. As capital and trade volume grow, these are the ordered scaling steps — implement them in sequence, not all at once.

Level 1 (Current): Single machine, 1–4 trades/day, $50–500 at risk - Everything in this guide - SQLite database, local logs, manual trade review - VPS or laptop sufficient

Level 2: Higher volume, $500–2000 at risk - Replace SQLite with PostgreSQL for concurrent writes - Add position tracking API (avoid manual P&L reconciliation) - Move daemons to a dedicated VPS ($4–6/month) for 24/7 uptime - Implement portfolio-level risk limits (not just per-trade)

Level 3: Multi-city simultaneous positions, $2000+ at risk - Dedicated cloud infrastructure (AWS/GCP) - Redis for real-time state (faster than SQLite for concurrent reads) - Separate services for scanner, executor, and monitor - Automated execution with human approval for positions >$200

Capital Allocation Optimization

The Kelly Criterion gives you the mathematically optimal position size. Half-Kelly (50% of Kelly) gives you a practical, drawdown-resistant version:

```
Kelly fraction = (p × b - (1 - p)) / b

Where:
  p = probability of winning (estimated fair value)
  b = net odds (what you win per dollar risked, i.e., (1-price)/price)

Half-Kelly = Kelly fraction × 0.5
```

As your sample size grows, your estimated fair values become more precise, and your Kelly calculations become more trustworthy. With 15 trades (February), the estimate is rough. With 150 trades, it's meaningful.

Dynamic Kelly adjustment based on recent performance:

```python
def dynamic_kelly(edge: float, win_rate_30day: float, bankroll: float,
                  base_max: float = 300.0) -> float:
    """
    Adjust position size based on recent win rate.

    Poor recent performance → reduce sizing
    Strong recent performance → allow full sizing

    This prevents the ruin scenario where a losing streak
    causes progressively larger bets to "make it back."
    """
```

```
    # Base Kelly
    kelly_fraction = edge / (1 - edge)  # Simplified for binary markets
    half_kelly = kelly_fraction * 0.5

    # Performance modifier
    if win_rate_30day >= 0.60:
        modifier = 1.0    # Full sizing
    elif win_rate_30day >= 0.50:
        modifier = 0.75   # Moderate sizing
    elif win_rate_30day >= 0.40:
        modifier = 0.50   # Conservative
    else:
        modifier = 0.25   # Minimal (in drawdown)

    position = min(bankroll * half_kelly * modifier, base_max)
    return round(position, 2)
```

Expanding Beyond Six Cities

The system supports any METAR station that maps to a Polymarket weather market. Adding a new city requires:

1. **Identify the resolution station** — check the specific market contract
2. **Study the meteorology** — what patterns affect this station?
3. **Add to stations.json** — include city, station, and any gate-relevant notes
4. **Paper trade for 10+ days** — validate your understanding before real capital
5. **Update the gate config** — does this city need a sea breeze or marine layer gate?

High-potential cities to add (in approximate priority):

City	Station	Primary Edge	Complexity
Phoenix	KPHX	Extreme heat brackets in summer	LOW
Denver	KDEN	Altitude effects, rapid changes	MEDIUM
Las Vegas	KLAS	Dry airmass, clean diurnal cycle	LOW
Houston	KHOU	Gulf moisture, complex sea breeze	HIGH
Boston	KBOS	Atlantic effects, fog patterns	HIGH

Model Improvement: The Compound Edge

The cusp model as implemented is version 1. Here's the research roadmap for improving edge accuracy:

v2.0: Multi-observation trend fitting Instead of linear trend detection (warming/cooling/stalling), fit a regression model to the past 6 hours of observations. The slope and curvature of the trend curve gives better peak timing estimates.

v2.1: Humidity adjustment Very high dewpoint temperatures cause the effective temperature-to-WU-display relationship to differ slightly from the simple rounding for-

mula. Integrating dewpoint into the safety margin calculation adds precision in humid summer conditions.

v2.2: Historical peak timing Build a database of historical temperature peaks by city and month. "Atlanta temperature peaks between 2:00 and 3:30 PM in March" — data-driven rather than eyeball. This improves early-entry timing.

v3.0: Ensemble forecasting Pull from multiple NWS models (GFS, HRRR, NAM) and average their projections. Ensemble divergence (models disagree) = higher uncertainty = reduce sizing. Ensemble convergence (models agree) = more confidence. The NWS NBM blend does some of this already, but raw ensemble access provides more information.

VPS Deployment Guide

For 24/7 operation without keeping your laptop running:

```
# On your local machine: set up VPS access
ssh-keygen -t ed25519 -C "trading-bot"
ssh-copy-id user@your-vps-ip

# On the VPS: install dependencies
sudo apt update && sudo apt install -y python3 python3-pip python3-venv tmux git

# Clone and configure
git clone https://github.com/YOUR_REPO/polymarket-trading-bot.git ~/trading-bot
cd ~/trading-bot
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
# Edit .env with your API keys

# Run installation verification
python3 -c "
from src.wu_rounding import WURoundingCalculator
calc = WURoundingCalculator()
print('  VPS installation verified')
"

# Add cron jobs (same as local setup)
crontab -e
# (paste the cron schedule from Part 10)

# Start daemons
bash start_trading.sh

# Monitor from your laptop
ssh user@your-vps-ip "tmux -S ~/.tmux/sock ls"
```

```
ssh user@your-vps-ip "tmux -S ~/.tmux/sock attach -t metar-daemon"
```

Recommended VPS providers: - **Hetzner Cloud (CX11):** ~€4.15/month. Best value, reliable, European DC - **DigitalOcean Basic:** ~$4/month. Simple interface, good US latency - **Vultr:** ~$2.50/month. Cheapest option, slightly lower reliability

For weather trading, low latency matters more than raw compute. Pick a US East Coast datacenter (New York or Atlanta) to minimize the round-trip time to `aviationweather.gov`.

Performance Benchmarks

After proper setup, these are the latency targets for each system component:

Component	Target Latency	Acceptable Maximum
METAR fetch (aviationweather.gov)	< 1 second	3 seconds
T-group parse	< 1 millisecond	10 milliseconds
Gate evaluation (all 4 gates)	< 5 milliseconds	50 milliseconds
Cusp model calculation	< 10 milliseconds	100 milliseconds
Wethr.net REST call	< 2 seconds	5 seconds
Wethr.net push delivery	< 30 seconds	60 seconds
SQLite decision log write	< 50 milliseconds	500 milliseconds

End-to-end: METAR post → trading decision → logged alert: < 15 seconds.

If you're seeing consistently higher latency, check: 1. Internet connection (run `ping aviationweather.gov`) 2. VPS geographic location (use a US East Coast server) 3. Python process health (check daemon logs)

Part 15: Reference Appendices

Appendix A: Complete Configuration Files

stations.json

```
{
  "version": "1.0",
  "description": "Canonical station map for Polymarket weather trading",
  "warning": "Station codes matter. Wrong station = wrong data = wrong trade.",
  "stations": {
    "KATL": {
      "city": "Atlanta",
      "airport": "Hartsfield-Jackson Atlanta International",
      "state": "GA",
      "timezone": "America/New_York",
      "coordinates": {"lat": 33.6407, "lon": -84.4277},
```

```
    "notes": [
      "Watch for late-afternoon cloud break warming events",
      "Urban heat island: 1-2°F warm bias vs surrounding areas",
      "Cold fronts can drop temp 5-8°F in 30 minutes"
    ],
    "gates_applicable": ["forecast_divergence", "polar_vortex"],
    "wu_history_url": "https://www.wunderground.com/history/daily/us/ga/atlanta/KATL"
  },
  "KDAL": {
    "city": "Dallas",
    "airport": "Dallas Love Field",
    "state": "TX",
    "timezone": "America/Chicago",
    "coordinates": {"lat": 32.8471, "lon": -96.8514},
    "notes": [
      "CRITICAL: This is KDAL (Love Field), NOT KDFW (DFW Airport)",
      "KDAL and KDFW can differ by 3-5°F on the same day",
      "30°F+ diurnal swings common in spring",
      "Peak temp: 2-4 PM local in winter, 3-5 PM in summer"
    ],
    "gates_applicable": ["forecast_divergence", "polar_vortex"],
    "wu_history_url": "https://www.wunderground.com/history/daily/us/tx/dallas/KDAL"
  },
  "KMIA": {
    "city": "Miami",
    "airport": "Miami International",
    "state": "FL",
    "timezone": "America/New_York",
    "coordinates": {"lat": 25.7959, "lon": -80.2870},
    "notes": [
      "Sea breeze: S wind (160-200°) + gusts  15kt = temp cap ~79°F",
      "Sea breeze typically activates 1-3 PM EST",
      "Low diurnal range (8-12°F) vs continental cities",
      "Resolves on station temp, not heat index"
    ],
    "gates_applicable": ["sea_breeze", "forecast_divergence"],
    "wu_history_url": "https://www.wunderground.com/history/daily/us/fl/miami/KMIA"
  },
  "KSEA": {
    "city": "Seattle",
    "airport": "Seattle-Tacoma International",
    "state": "WA",
    "timezone": "America/Los_Angeles",
    "coordinates": {"lat": 47.4502, "lon": -122.3088},
    "notes": [
      "Marine layer: W-NW wind (250-320°) = cooling cap, models overestimate by 2-4°F",
      "Pacific air masses create persistent temp ceilings",
      "Winter inversions trap cold air in Puget Sound basin",
```

```
        "Models frequently overshoot winter highs"
      ],
      "gates_applicable": ["marine_layer", "forecast_divergence"],
      "wu_history_url": "https://www.wunderground.com/history/daily/us/wa/seattle/KSEA"
    },
    "KLGA": {
      "city": "New York City",
      "airport": "LaGuardia Airport",
      "state": "NY",
      "timezone": "America/New_York",
      "coordinates": {"lat": 40.7772, "lon": -73.8726},
      "notes": [
        "Coastal effects from Long Island Sound",
        "Urban heat island strongest at night",
        "W-NW wind brings marine influence similar to Seattle",
        "Wind channeling through Manhattan creates local effects"
      ],
      "gates_applicable": ["marine_layer", "forecast_divergence", "polar_vortex"],
      "wu_history_url": "https://www.wunderground.com/history/daily/us/ny/new-york-city/KLGA"
    },
    "KORD": {
      "city": "Chicago",
      "airport": "O'Hare International",
      "state": "IL",
      "timezone": "America/Chicago",
      "coordinates": {"lat": 41.9742, "lon": -87.9073},
      "notes": [
        "Lake Michigan effect: E-NE winds in spring/summer cap temps",
        "Most predictable patterns - 100% win rate in our initial data",
        "Polar vortex events create multi-day temp locks",
        "Runway heating in summer: station temp 1-2°F above surroundings"
      ],
      "gates_applicable": ["forecast_divergence", "polar_vortex"],
      "wu_history_url": "https://www.wunderground.com/history/daily/us/il/chicago/KORD"
    }
  }
}
```


brackets.json


```
{
  "version": "1.0",
  "description": "Active bracket definitions for Polymarket weather markets",
  "conventions": {
    "bracket_format": "[lower, upper] - inclusive on both ends",
    "resolution": "WU displayed high temperature, whole Fahrenheit",
    "station": "See stations.json for the specific ICAO station per city"
  },
```

```
  "standard_brackets": [
    {"lower": 30, "upper": 39, "label": "30-39°F"},
    {"lower": 40, "upper": 49, "label": "40-49°F"},
    {"lower": 50, "upper": 59, "label": "50-59°F"},
    {"lower": 60, "upper": 69, "label": "60-69°F"},
    {"lower": 70, "upper": 79, "label": "70-79°F"},
    {"lower": 80, "upper": 89, "label": "80-89°F"},
    {"lower": 90, "upper": 99, "label": "90-99°F"}
  ],
  "narrow_brackets": [
    {"lower": 56, "upper": 57, "label": "56-57°F"},
    {"lower": 58, "upper": 59, "label": "58-59°F"},
    {"lower": 64, "upper": 65, "label": "64-65°F"},
    {"lower": 66, "upper": 67, "label": "66-67°F"},
    {"lower": 70, "upper": 71, "label": "70-71°F"},
    {"lower": 78, "upper": 79, "label": "78-79°F"},
    {"lower": 80, "upper": 81, "label": "80-81°F"}
  ],
  "extreme_brackets": [
    {"lower": -10, "upper": 19, "label": "Below 20°F"},
    {"lower": 100, "upper": 115, "label": "100°F+"}
  ]
}
```

Full .env.example

```
#
# Trading System - Full Environment Configuration Template
# Copy to .env and fill in your actual values.
# NEVER commit .env to git.
#

#   Data Sources

# Wethr.net Pro API (REQUIRED - resolution-matching data)
# Sign up: https://wethr.net/pro
WETHR_API_KEY=

# Anthropic Claude API (REQUIRED - AI fair value estimation)
# Get key: https://console.anthropic.com
ANTHROPIC_API_KEY=
CLAUDE_MODEL=claude-3-haiku-20240307

#   Polymarket Trading

# Wallet private key (from MetaMask or similar)
# Fund wallet with USDC on Polygon network
POLYMARKET_PRIVATE_KEY=
```

```
POLYMARKET_FUNDER=
POLYMARKET_SIGNATURE_TYPE=1

# Polymarket API endpoints (don't change unless they update)
CLOB_ENDPOINT=https://clob.polymarket.com
GAMMA_ENDPOINT=https://gamma-api.polymarket.com

#   Notifications

# Telegram bot (OPTIONAL - for Telegram push alerts)
# Create bot: https://t.me/BotFather
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHAT_ID=

#   Trading Parameters

# Minimum edge % to consider a trade (recommended: 8-12%)
MIN_EDGE_PERCENT=8.0

# Maximum single position in USDC
MAX_BET_SIZE=5.0

# Maximum total capital at risk across all positions
MAX_TOTAL_RISK=100.0

# Market scanner interval (minutes)
SCAN_INTERVAL_MINUTES=10

# Minimum market liquidity required (USDC)
MIN_LIQUIDITY=5000

# Minimum 24-hour trading volume required (USDC)
MIN_VOLUME=2000

#   Safety Parameters

# Absolute minimum safety margin (never trade below this)
# This is the Feb 15 Rule - do not lower it
MIN_SAFETY_MARGIN_F=0.3

# Maximum trades per day
MAX_TRADES_PER_DAY=4

# Circuit breaker: consecutive losses before pause
CIRCUIT_BREAKER_LOSSES=3

#   Logging
```

```
# Log level: DEBUG, INFO, WARNING, ERROR
LOG_LEVEL=INFO

# Log file paths
LOG_DIR=./logs
DECISIONS_DB=./data/decisions.db
METAR_ALERTS_FILE=/tmp/metar_alerts.jsonl
```

Appendix B: Complete Prompt Library

These are the exact prompts used by the AI trading analyst (OpenClaw/Claude) for each analytical task.

Prompt 1: Morning Market Analysis

```
You are a professional weather trading analyst specializing in Polymarket temperature bracket m

Today is {DATE}. Your task is to produce a pre-market briefing for the 6 target cities.

## Weather Data Available

### WU Highs (overnight - resolution-matching)
{WU_HIGHS_TABLE}

### NWS Forecast Highs (today projected)
{NWS_FORECASTS_TABLE}

### Current METAR Observations
{METAR_CURRENT_TABLE}

### Active Weather Patterns
{WEATHER_PATTERNS}

## Instructions

For each city:
1. State today's most likely temperature range (high and low) and your confidence level
2. Identify any active weather gates that could block trades (sea breeze, marine layer, forecas
3. Identify the 1-2 most interesting brackets given today's expected temperatures
4. Flag any city where the forecast seems unreliable

## Output Format

### {CITY} ({STATION})
**Expected Range:** {LOW}°F - {HIGH}°F (Confidence: {HIGH/MEDIUM/LOW})
**Active Gates:** {GATES or "None"}
```

```
**Top Bracket:** {BRACKET} - {REASON}
**Data Quality:** {QUALITY ASSESSMENT}

---

Provide the briefing now. Be specific and direct - this is operational guidance, not weather ec
```

Prompt 2: Real-Time Trade Opportunity Assessment

```
You are analyzing a potential weather market trade in real time.

## Market Data
- City: {CITY} ({STATION})
- Bracket: {LOWER}-{UPPER}°F
- Direction: {YES/NO}
- Current market price: {PRICE} (i.e., {PRICE*100}¢)
- Market liquidity: ${LIQUIDITY}

## Temperature Data (Right Now)
- METAR T-group: {TGROUP} ({TEMP_C}°C = {TEMP_F}°F)
- METAR observation time: {OBS_TIME} ({AGE} minutes ago)
- WU currently displays: {WU_DISPLAY}°F
- WU daily high (so far): {WU_HIGH}°F
- NWS forecast high: {NWS_HIGH}°F
- Temperature trend (last 3 readings): {TREND}
- 5-minute observations (recent): {READINGS}

## Safety Analysis
- WU rounding safety margin: {MARGIN}°F
- Margin risk level: {RISK_LEVEL}
- Gate status: {GATE_STATUS}

## Task
1. Estimate the true probability that the daily high will be within [{LOWER}, {UPPER}]°F
2. Calculate the implied edge if this probability is correct
3. Identify any concerns or red flags
4. Give a clear YES (trade it) / NO (skip it) / WAIT (more data needed) recommendation with siz

## Output Format
**Fair value estimate:** {PROBABILITY}
**Implied edge:** {EDGE}%
**Recommendation:** {YES/NO/WAIT}
**Position size:** ${SIZE} ({REASON})
**Key risk:** {RISK}
**Confidence:** {HIGH/MEDIUM/LOW}

Be direct. Assume the reader will act on this immediately.
```

Prompt 3: Post-Trade Outcome Analysis

```
A weather trade has resolved. Analyze the outcome and extract lessons.

## The Trade
- City: {CITY}
- Bracket: {LOWER}-{UPPER}°F
- Direction: {YES/NO}
- Entry price: {ENTRY_PRICE}
- Position size: {SIZE_USDC}
- Entry time: {ENTRY_TIME}

## At Entry
- METAR temp: {ENTRY_TEMP_F}°F
- Safety margin: {ENTRY_MARGIN}°F
- NWS forecast: {ENTRY_FORECAST}°F
- Edge estimate: {ENTRY_EDGE}%
- Confidence: {ENTRY_CONFIDENCE}

## Resolution
- WU displayed high: {RESOLUTION_TEMP}°F
- Outcome: {WIN/LOSS}
- P&L: {PNL_USDC}

## Analysis Questions
1. Was the trade thesis correct? (Was the edge real, regardless of outcome?)
2. Was the position size appropriate for the actual safety margin?
3. Were any weather gates triggered that should have blocked this trade?
4. What would you do differently?

## Output Format
**Thesis assessment:** {WAS THE REASONING SOUND?}
**Sizing assessment:** {WAS THE SIZE APPROPRIATE?}
**Gate assessment:** {SHOULD ANY GATE HAVE BLOCKED THIS?}
**Key lesson:** {ONE SPECIFIC THING TO IMPROVE}
**Pattern update:** {ANY PATTERN TO ADD TO MEMORY.md?}
```

Prompt 4: Cusp Model Interpretation

```
You are interpreting real-time temperature data to determine whether a cusp condition exists.

## Raw Data
- Station: {STATION} ({CITY})
- Latest METAR T-group: {TGROUP} ({TEMP_C}°C = {TEMP_F}°F, {AGE} min ago)
- Recent 5-minute whole-degree readings (oldest → newest): {READINGS}
- Current WU daily high: {WU_HIGH}°F
- NWS forecast high: {NWS_HIGH}°F
- Current trend: {TREND}
```

```
## Task
Analyze whether:
1. The temperature is oscillating near a WU rounding boundary (cusp condition)
2. The peak has been reached (temperature stalling or declining)
3. Further warming is likely before market resolution

## Output Format
**Cusp condition:** {YES/NO/UNCERTAIN}
**Estimated actual temp:** {TEMP}°F ± {UNCERTAINTY}°F
**WU will display:** {WU_DISPLAY}°F (estimated)
**Peak assessment:** {REACHED/NOT YET/UNKNOWN}
**Safety margin to bracket [X-Y]:** {MARGIN}°F
**Action:** {HOLD/EXIT/WAIT/ENTRY OK}
**Reasoning:** {2-3 sentences}
```

Prompt 5: Evening Review

```
Conduct an end-of-day review for the weather trading operation.

## Today's Trades
{TRADES_TABLE}
(Columns: City, Bracket, Direction, Price, Size, Safety Margin, Gates, Outcome, P&L)

## Today's Metrics
- Trades executed: {COUNT}
- Win rate: {WIN_RATE}%
- Total P&L: ${PNL}
- Largest win: ${MAX_WIN}
- Largest loss: ${MAX_LOSS}

## Rolling Performance (last 20 trades)
- Win rate: {ROLLING_WIN_RATE}%
- Total P&L: ${ROLLING_PNL}
- Circuit breaker status: {ACTIVE/INACTIVE}

## Questions to Answer
1. Were today's position sizes appropriate given the safety margins?
2. Did any weather gates fire correctly? Incorrectly?
3. Is the rolling win rate above or below 50%? What does this imply?
4. What are tomorrow's top 2-3 opportunities (based on tonight's forecast data)?

## Output Format
**Today's assessment:** {1-2 SENTENCES}
**Sizing discipline:** {GOOD/NEEDS WORK + SPECIFIC EXAMPLE}
**Gate performance:** {GATES FIRED + ACCURACY}
**Rolling status:** {HEALTHY/MONITOR/CIRCUIT BREAKER ALERT}
**Tomorrow's pre-game:** {TOP 2-3 SETUPS WITH SPECIFIC BRACKETS}
```

Appendix C: System Architecture Diagrams

Full Stack Component Diagram

```
                    TRADING SYSTEM v1.0

LAYER 1: DATA SOURCES       LAYER 2: PROCESSING

aviationweather.gov         metar_fetcher.py
   METAR raw text            parse_tgroup() → 0.1°C precision
   Every :53 UTC
                            metar_cusp_model.py
Wethr.net Pro API             detect_oscillation()
   REST: WU-logic high       estimate_current_temp()
   REST: NWS forecast        recommend_action()
   SSE: Push stream
                            wu_rounding.py
Polymarket Gamma API           calculate_safety_margin()
   /markets                    (the Feb 15 prevention)
   /prices
   /trades                 weather_gates.py
                               _check_sea_breeze()
METAR Daemon                   _check_marine_layer()
(tmux session)                 _check_forecast_divergence()
   2s polls at :52-:58       _check_polar_vortex()


LAYER 3: DECISION ENGINE    LAYER 4: EXECUTION & LOGGING

best_trades_query.py        position_sizer.py
   Market scanner            modified_half_kelly()
     Edge calculation           margin_tiering()
     Opportunity ranking        time_of_day_penalty()

signal_validator.py         trade_executor.py
   Multi-source check        Pipeline orchestrator
     2+ sources required        Gate → Cusp → Size → Execute

OpenClaw AI Agent           decisions.db (SQLite)
   Fair value prompt         Every decision logged
     Pattern memory             Pattern queries
     Alert routing              P&L tracking


LAYER 5: HUMAN INTERFACE
```

```
Telegram (phone)
   Morning briefing (8 AM)
   URGENT exit alerts (sub-5-second delivery)
   Opportunity alerts (>15% edge)
   Evening review (9 PM)

Terminal (laptop/VPS)
   python3 best_trades.py        → Quick opportunity scan
   python3 wethr_summary.py      → All 6 cities status
   python3 src/metar_aggressive_daemon.py → Start METAR monitor
   sqlite3 data/decisions.db     → Query performance data
```

Trade Decision Flow

```
New METAR observation detected


   Parse T-group
   (0.1°C precision)




   Update cusp model                    Log to metar_log
   - Oscillation detection              (decisions.db)
   - Trend assessment
   - WU margin calculation


   Gate check (all 4)

     BLOCKED?  YES   STOP: Log skipped decision

           NO


   Market scanner
   - Edge calculation
   - Fair value vs price

   EDGE < 8%?  YES   STOP: No trade

           NO


   Signal validation
```

```
   - Count confirming sources

   < 2 SOURCES?  YES   STOP: Insufficient confirmation

          NO


   Position sizer
   - Safety margin → size tier
   - Dynamic Kelly adjustment
   - Portfolio limit check

   SIZE = $0?  YES   STOP: Margin too thin

          NO


   Log decision to DB (BEFORE execution)


   Push opportunity alert → Telegram


   Human reviews → Approves/Rejects

            (APPROVED)
   Execute trade via Polymarket API


   Update DB with execution confirmation


   Monitor position (heartbeat every 30 min)


   Resolution → Update outcome in DB
```

Cron + Daemon Timing Diagram

```
DAILY TIMELINE (all times EST)

12:00 AM   Midnight cleanup (log rotation, archive)

 6:00 AM
 7:00 AM
 7:45 AM   Morning prep cron: fetch overnight WU highs, NWS v1
 8:00 AM   Morning briefing push to Telegram
```

```
 9:00 AM    [Daemons wake if VPS uptime] Check health cron
10:00 AM    Market opens - scanner activates (every 10 min)

10:30 AM    First heartbeat (30-min check)
11:00 AM    METAR window (:52-:58 UTC = :52-:58 AM EST+6... UTC-5)
              [Daemon aggressive polling begins - 2s intervals]

 NOON      Optimal entry window (early trades → morning entries)

 1:00 PM    NWS typically updates (v2-v4 of today's forecast)
              → Scanner picks up revision → Telegram alert if gap
 2:00 PM    Peak temperature window for most cities
 3:00 PM    Late entry warning: confidence cap at 90%
 4:00 PM    Position size penalty begins (-30%)
 5:00 PM    No new entries unless margin > 1.5°F

 6:00 PM    Most markets approaching resolution
 7:00 PM
 8:00 PM
 9:00 PM    Evening review push to Telegram
              Market resolution checks

10:00 PM    Scanner deactivates
11:59 PM    Midnight cleanup (next day)


METAR DAEMON DETAIL (each UTC hour during market hours)

:00     Sleep between windows
:10
:20
:30
:40
:51     Wake up (1 min before window)
:52     Begin 2-second polling
:53     New METAR observation detected (typical)
          T-group parsed (0.1°C precision)
          Gate check run
          Cusp model updated
          Alert if margin breach
          Decision logged
:54     Continue polling (verifying no SPECI)
:55     Continue polling
:56     Continue polling
:57     Continue polling
:58     Final poll of window
:59     Window closed - sleep until :51 next hour
```

Appendix D: Troubleshooting Guide

"No T-group in METAR"

Not all METAR observations include the T-group. This happens with: - Some international stations (rare for US airports) - Station equipment issues - ASOS (automated stations) — they should always have T-groups, but occasionally drop them

Fix: Wait for the next observation. If 3 consecutive observations are missing T-groups, check if the station is reporting a different format (try raw METAR from NOAA).

"Wethr.net returning stale data"

The WU high might not update immediately after a new METAR posts. Wethr.net processes in near-real-time but has a 30–90 second ingestion delay.

Fix: Wait 2 minutes after a new METAR observation, then re-query. If still stale, check Wethr.net status page.

"Position sizer returning $0 but safety margin looks fine"

Check these in order: 1. Is `direction` set correctly ('YES' or 'NO')? Wrong direction inverts the margin calculation. 2. Is the actual temperature in the correct bracket? A margin of 0.45°F is fine if inside the bracket but dangerous if you're betting on a bracket you haven't entered yet. 3. Is the METAR age < 50 minutes? Stale METAR → uncertainty spike → reduced position.

"Gate triggering incorrectly"

Sea breeze false positive: Wind direction right but speed too low. Check `wind.gust` — if null (no gusts), the gate uses speed only. You can temporarily lower the gust threshold in the code.

Forecast divergence false positive: Scanner prediction is based on a stale NWS run. Run `python3 scripts/update_forecasts.py` to pull the latest NWS data before the gate re-evaluates.

"METAR daemon not detecting new observations"

1. Check if the daemon is running: `tmux -S ~/.tmux/sock ls`
2. Check the log: `tmux -S ~/.tmux/sock capture-pane -t metar-daemon -p | tail -30`
3. Verify internet access: `curl -s https://aviationweather.gov/api/data/metar?ids=KATL | head -5`
4. If METAR website is down: use Wethr.net's observation data as a fallback

“Trade executed at wrong price”

Polymarket order books can move fast. By the time your order executes, the price may be different from what the scanner quoted.

Prevention: Set a maximum acceptable price (slippage limit) in trade_executor.py. If market price moved more than 3 cents from scanner price, don’t execute — re-evaluate.

Appendix E: Glossary

Term	Definition
ASOS	Automated Surface Observing System — the automated weather sensors at most US airports
Bracket	A temperature range bet on Polymarket (e.g., “Will the high be 64–65°F?”)
CLOB	Central Limit Order Book — Polymarket’s matching engine for YES/NO share trades
Cusp	A temperature value near a WU rounding boundary, creating edge
Diurnal range	The difference between daily high and low temperature
Edge	The mathematical advantage: (fair value − market price) / market price
Gate	A trade-blocking rule that fires when a known failure pattern is detected
Half-Kelly	50% of the Kelly Criterion bet size — reduces volatility while maintaining edge capture
ICAO	International Civil Aviation Organization — the four-letter station code system
Kelly Criterion	The mathematically optimal fraction of bankroll to bet given an edge
Marine layer	Coastal fog/low cloud from Pacific or Atlantic air, causing temperature ceilings
METAR	Meteorological Aerodrome Report — hourly weather observation at airports
NWS	National Weather Service — provides free forecast data used as our prediction layer
Oscillation	When 5-minute METAR readings flip between adjacent values, indicating a cusp
Polar vortex	A large-scale air circulation that can push Arctic air into the continental US
Safety margin	Distance from current temperature to the nearest WU rounding threshold
Sea breeze	A coastal wind pattern driven by land-sea temperature contrast, creating a temperature cap

Term	Definition
SPECI	Special weather report — issued when conditions change significantly between regular METARs
SSE	Server-Sent Events — a protocol for real-time push data from servers
T-group	The temperature group in METAR remarks, providing 0.1°C precision
USDC	USD Coin — the stablecoin used for betting on Polymarket (on Polygon network)
WU	Weather Underground — the resolution oracle for most Polymarket weather markets

---

Appendix F: Quick Reference Card

Print this. Put it next to your screen on trading days.

```
        WEATHER TRADING QUICK REFERENCE

STATION MAP (memorize these or you'll lose money)
Atlanta  → KATL     Dallas   → KDAL (NOT KDFW!)
Miami    → KMIA     Seattle  → KSEA
NYC      → KLGA     Chicago  → KORD

POSITION SIZING BY SAFETY MARGIN
  1.0°F  → Full position (up to $300)
0.5-1.0  → Half position
0.3-0.5  → Quarter position
< 0.3°F  → $0 - NO TRADE (absolute rule)

GATE RULES
Sea breeze (Miami, S wind + gust  15kt): HARD BLOCK
Marine layer (SEA/NYC, W-NW wind):       CAUTION 50%
Forecast divergence (>1.5°F, 30+ min):  HARD BLOCK
Polar vortex (>20°F below avg, no trend): HARD BLOCK

ENTRY CHECKLIST
  2+ confirming data sources
  All gates checked
  WU rounding margin calculated
  Position sized from calculator
  Decision logged to DB before execution

CIRCUIT BREAKERS
3 consecutive losses → pause 24 hours
```

```
Single loss > $100 → mandatory review
Win rate < 50% / 20 trades → pause trading

KEY COMMANDS
python3 best_trades.py          Top 2 opportunities
python3 wethr_summary.py        All 6 cities status
tmux -S ~/.tmux/sock ls         Daemon health check
sqlite3 data/decisions.db       Query trade history
```

Final Word: The Operation You've Built

Fifteen trades. Four wins. Eleven losses. -$187. That's where this started.

Not because the edge wasn't there — it was. Weather Underground rounding creates real, quantifiable information asymmetries. METAR T-group data genuinely provides 30–60 minute advantages over consumer apps. The cusp model describes real physics.

The losses happened because of discipline gaps. Wrong station. Oversized position. Single data source. Panic exit. Sea breeze ignored. Forecast trusted past its expiration date.

Every one of those failures is now a component in the system you've built:

- Wrong station → `stations.json`, with explicit warnings
- WU rounding blindness → `wu_rounding.py`, with Feb 15 self-validation
- Panic exits → Oscillation detector, 3-reading confirmation rule
- Oversizing → Modified half-Kelly with margin tiering
- Single source → Two-source requirement in `signal_validator.py`
- Sea breeze ignored → Gate 1, hard block, no override
- Stale forecast trusted → Gate 3, divergence detection

The losses were expensive. The system they produced is worth more.

Now the system runs. You review opportunities. You approve trades that clear all gates, have 2+ confirming sources, and have position sizes calculated by the machine rather than your gut. The daemon watches for new METAR observations at 2-second resolution. The push stream delivers WU high updates the moment they happen. The decision database builds the pattern library that makes every future trade better-informed than the last.

This is the complete system. Build it. Test it. Trust the gates. And remember the first rule of weather trading: it's always cheaper to skip a good trade than to take a bad one.

— Obsidian, JSJ Consulting Version 1.0 — March 2026

© 2026 JSJ Consulting. All rights reserved. Licensed for personal use only. Do not redistribute, resell, or share without written permission.